common ITALY

IBM i

# Commitment control

*for everyone*

Marco Riva

www.markonetools.it

IBM CHAMPION 2021-2025 Power area

RPG IBM DB2 SQL

Ultimo aggiornamento: 12/03/2026

1

KEY POINTS

| Critical | Important | Nice to have |
|---|---|---|
| Gruppi attivazione<br>Ambito<br>Attivazione<br>Operazioni implicite | Livello isolamento<br>Concorrenza accesso<br>Monitoraggio | Deadlock<br>Savepoint<br>Soft commit<br>Giornali |

2

## Sommario


21/gen (1)
- Un ripasso di job e activation group
- Concetti base del controllo di sincronia e glossario
- Prerequisiti
- Attivare il controllo di sincronia
- Ambito del controllo di sincronia
- Perché usare il controllo di sincronia
- Livello di isolamento

4/feb (2)
- Commit e rollback impliciti
- Esempi di transazioni

18/feb (3)
- Concorrenza di accesso
- Cursori SQL vs commit/rollback
- Savepoint
- Monitoraggio del controllo sincronia
- Controllo di sincronia e voci di giornale
- Istruzioni SQL per i giornali


3

## Esempi e documentazione del corso

https://github.com/mk1tools/Commitment-control-tutorial.git

GitHub

4

# Controllo di sincronia

Concetti base

5

## Controllo sincronia (commitment control)

**COSA**

- consente di ***definire e consolidare*** un gruppo di modifiche sul database come **unità logica di lavoro** (LUW) = **transazione**

**COME**

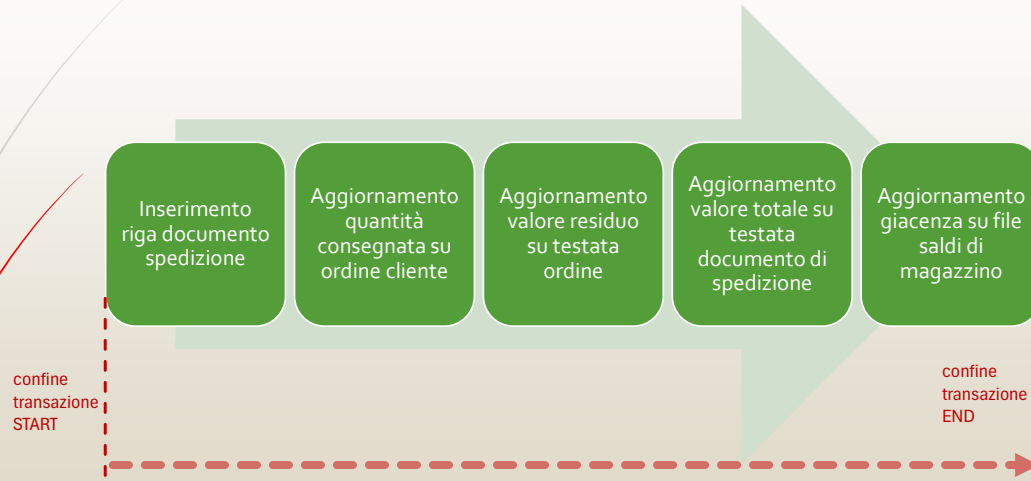
- ***transazione***
- è un gruppo di modifiche a uno o più file di database che dal punto di vista dell'utente appaiono come una singola modifica

**PERCHÉ**

- ***integrità e coerenza del dato***
- Assicura che l'intero gruppo di modifiche apportate in una transazione o vengano tutte consolidate sul db (*commit*) o tutte annullate (*rollback*)

6

## Esempio transazione




Ogni step può essere eseguito da uno o più programmi, quindi la transazione supera i confini del programma

7

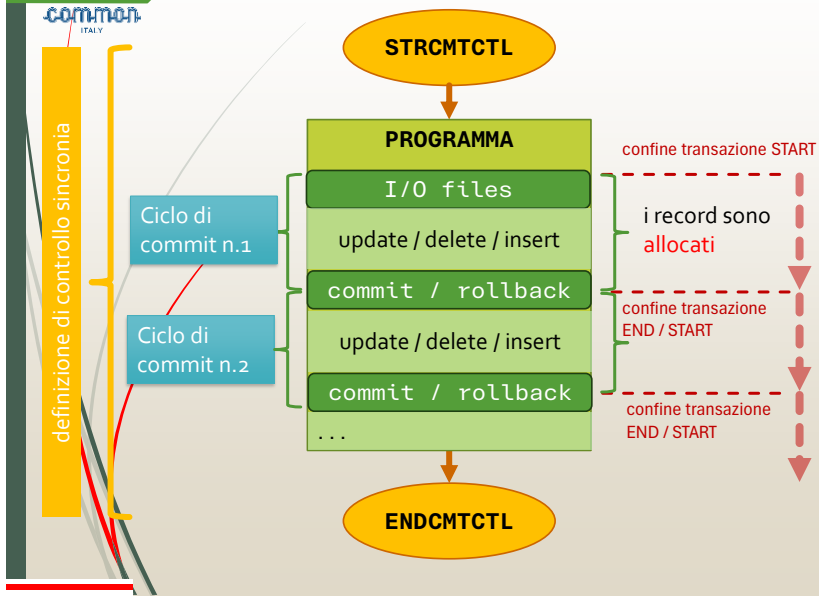
## Commit e Rollback

- **COMMIT**
  rende permanenti (= **consolida**) tutte le modifiche eseguite sui record del db (update/delete/insert) e sugli oggetti db (create/alter/drop...) dal precedente commit/rollback

- **ROLLBACK**
  **rimuove** tutte le modifiche eseguite sui record del db (update/delete/insert) e sugli oggetti db (create/alter/drop...) dal precedente commit/rollback

8

## Controllo di sincronia – schema logico




- La prima operazione di I/O e i successivi commit/rollback definiscono i confini della transazione
- per le applicazioni SQL il controllo di sincronia viene attivato automaticamente nell'ambito dell'activation group
- Un ciclo di commit è il tempo che intercorre tra due confini di transazione.
  Il sistema assegna un identificatore ad ogni ciclo di commit

9

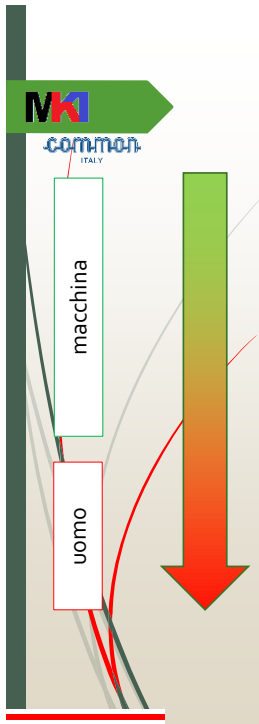
## Controllo sincronia: prerequisiti

- I files devono essere registrati sullo **stesso giornale** con entrambe le immagini before e after.
  Se il file è registrato solo con immagine after, il sistema registrerà anche l'immagine before
- I/O nativo: avviare definizione del controllo sincronia (STRCMTCTL) e definire i file con keyword **COMMIT**
- I/O SQL: impostare il livello di isolamento (**set option commit**) diverso da *NONE

`create schema` o `create collection` creano automaticamente una nuova libreria con il giornale di default QSQJRN e qualsiasi tabella creata nella libreria viene registrata sul giornale

Se si crea la libreria con CRTLIB occorre creare manualmente ricevitore e giornale ed attivare la registrazione con STRJRNLIB

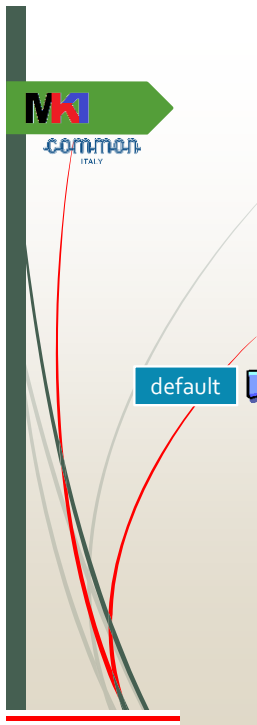
10




## Perché lo si teme?

- richiede che tutti i file siano registrati sullo stesso giornale. Quindi si evita per timore di decadimento di performance e di occupazione spazio disco
- prima di RPG IV era macchinoso l'uso opzionale del controllo di sincronia nei programmi
- prima di ILE il controllo di sincronia agiva solo a livello di job e non dell'activation group
- **poiché i file sono registrati su giornale richiede maggior attenzione nella manipolazione di oggetti e librerie**
- **complessità del disegno applicativo per definire correttamente le transazioni e gli activation group**

11




## Ambito delle transazioni

- L'ambito (=**scope**) delle transazioni può essere a livello
  - **Lavoro**: ogni programma chiamato dopo `STRCMTCTL CMTSCOPE(*JOB)` in esecuzione in qualsiasi gruppo di attivazione che non abbia un controllo di sincronia specifico del proprio gruppo di attivazione userà il controllo di sincronia a livello di lavoro
  - **Gruppo di attivazione**: `STRCMTCTL CMTSCOPE(*ACTGRP)` oppure applicazione SQL con `SET OPTION COMMIT` diverso da `*NONE`. Solo i programmi in esecuzione nel gruppo di attivazione useranno il controllo di sincronia specifico di quel gruppo
  - **Transazione**: avviati con XA APIs for Transaction Scoped Locks. Questa API è utilizzata per associare la definizione del controllo sincronia a uno specific thread o a una connessione SQL e non all'activation group

12

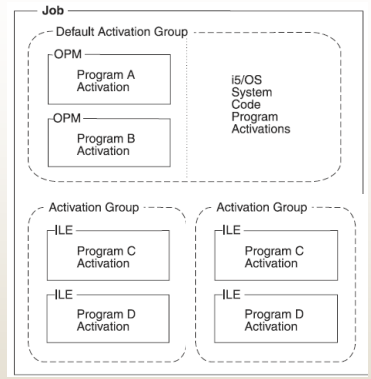
# Job e activation group

Un breve ripasso

13

## Job e activation group: program isolation

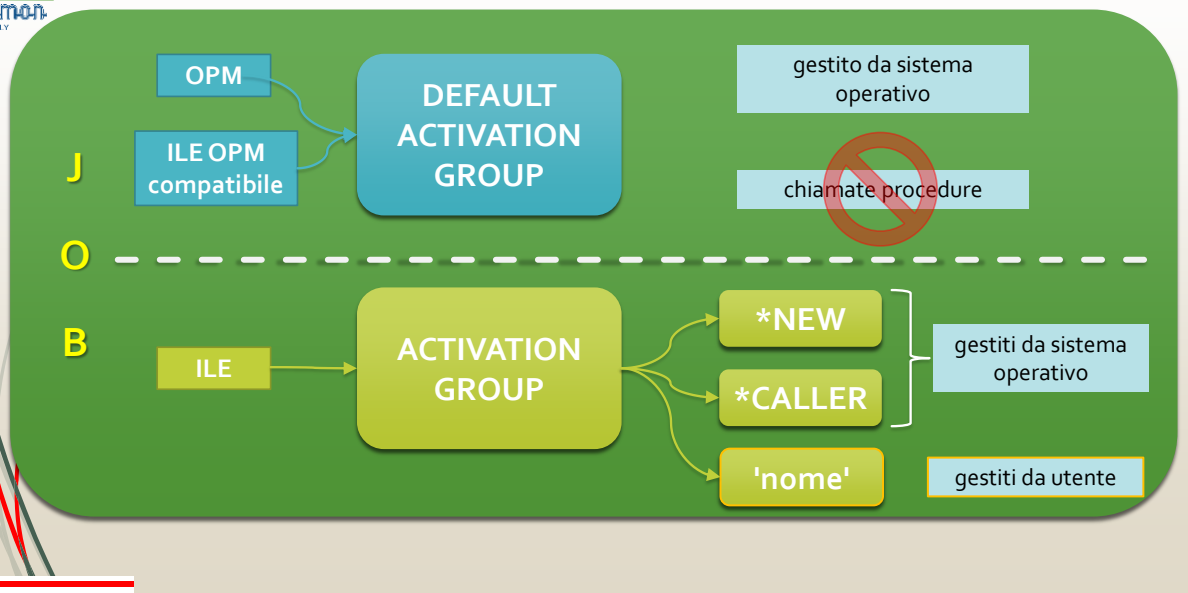
- L'**activation group** contiene le **risorse** necessarie ad eseguire il programma. Le risorse sono:
  - Static program variables
  - Dynamic storage
  - Temporary data management resources
  - ODP (Open Data Path)
  - Commitment definitions
  - SQL cursors
  - HFS (Hierarchical File System)
- All'avvio di un job, *automaticamente* vengono creati 2 activation group di **default** usati da tutti i programmi OPM e ILE OPM compatible




Fonte IBM

14

## Activation group in sintesi

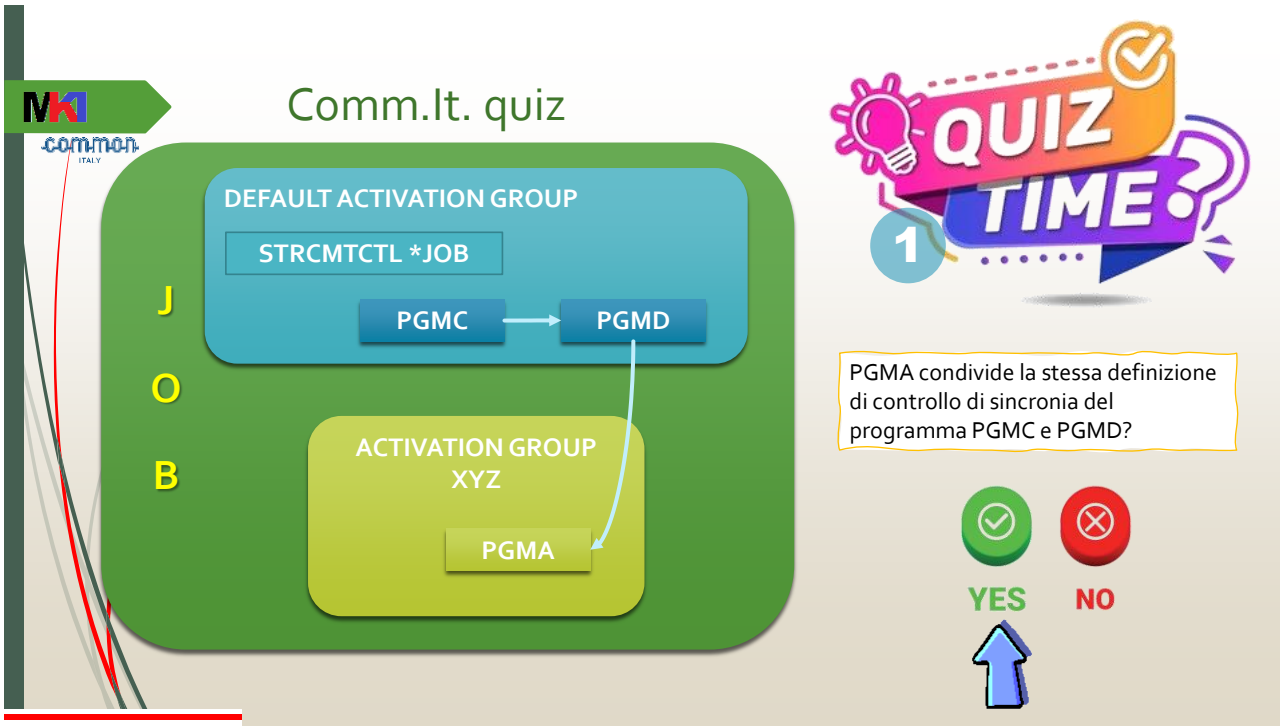



15

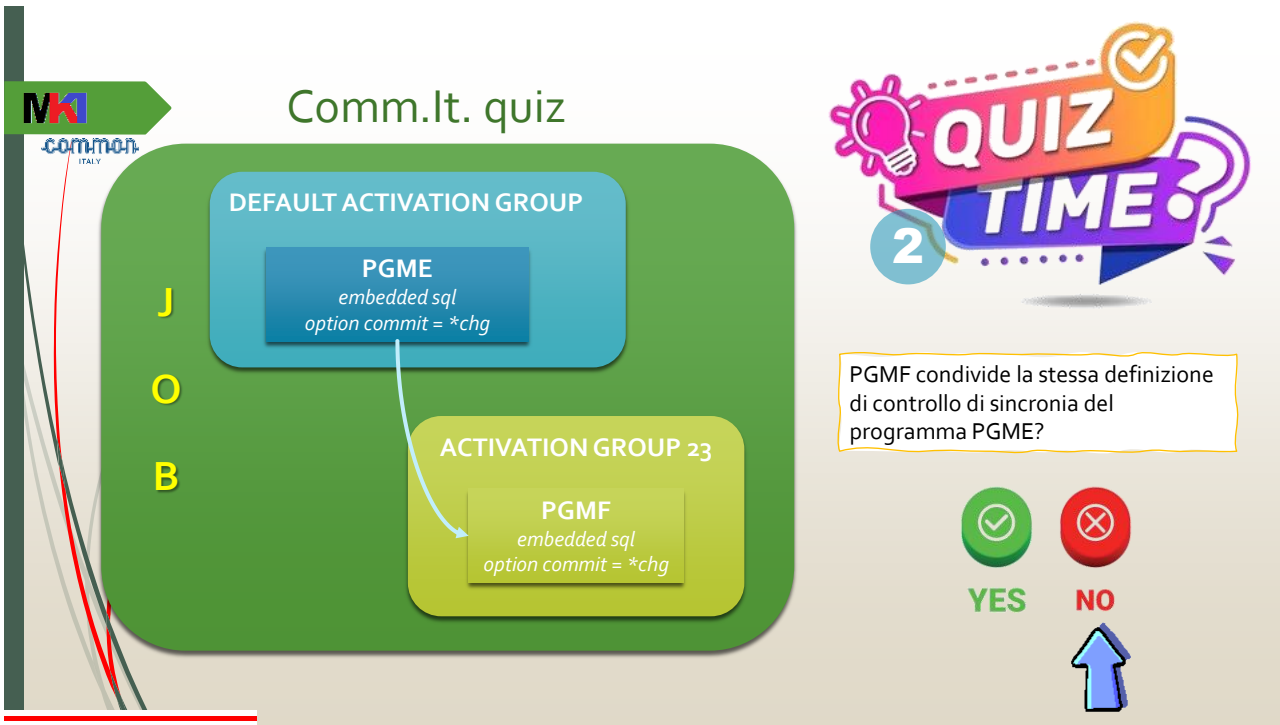
## Activation group: comandi principali

- **RCLACTGRP**: libera le risorse di un AG nominale
- **CRTSQLRPGI** parametro **CLOSQLCSR**: determina quando viene chiuso automaticamente un cursore SQL. `*ENDACTGRP` o `*ENDMOD`.
- **OVR**... parametro **OVRSCOPE**: determina l'ambito di validità della sostituzione. `*ACTGRPDFN` o `*CALLLVL` o `*JOB`.
- **DLTOVR** parametro **LVL**: determina quale livello di sostituzioni eliminare. `*ACTGRPDFN` o `*` o `*JOB`.

16

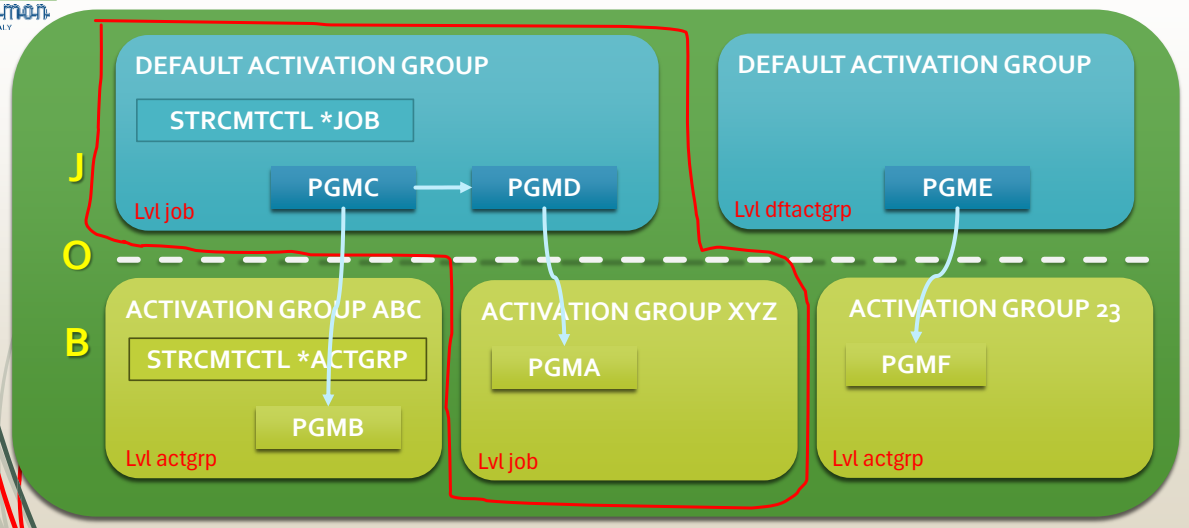

Comm.lt. quiz
DEFAULT ACTIVATION GROUP
STRCMTCTL *JOB
PGMC
PGMD
J
O
B
ACTIVATION GROUP
XYZ
PGMA
QUIZ
TIME?
1
PGMA condivide la stessa definizione di controllo di sincronia del programma PGMC e PGMD?
YES
NO

17


Comm.lt. quiz
DEFAULT ACTIVATION GROUP
PGME
embedded sql
option commit = *chg
J
O
B
ACTIVATION GROUP 23
PGMF
embedded sql
option commit = *chg
QUIZ
TIME?
2
PGMF condivide la stessa definizione di controllo di sincronia del programma PGME?
YES
NO

18

## Controllo sincronia - isolamento transazioni/1




✳ PGMA.RPGLE, PGMB.RPGLE, PGMC.RPGLE, PGMD.RPGLE, PGME.SQLRPGLE, PGMF.SQLRPGLE

19

## Controllo sincronia - isolamento transazioni/2

*Annotazioni allo schema precedente*

- PGMC avvia STRCMTCTL a livello *JOB
- i programmi PGMD e PGMA chiamati da PGMC ereditano la definizione di commit a livello job
- il programma PGMB avvia STRCMTCTL a livello *ACTGRP quindi viene creata una definizione di commit a livello dell'activation group ABC
- il PGME (embedded SQL) nel dftactgrp non avvia esplicitamente STRCMTCTL quindi si crea una definizione implicita di commit a livello *DFTACTGRP
- il PGMF (embedded SQL) chiamato da PGME apre un nuovo activation group e implicitamente crea una nuova definizione di commit a livello dell'activation group

20

# Demo - CALL PGMC/1

WRKCMTDFN

PGMC

```
DSPLY  Avviato contr.sincr. *JOB. Invio per proseguire
```

```
Opz  Definiz. commit   N° lav.   Uten. lav.   Nome lav.   Sottopr.   Utente
     *JOB              460667    MRIVA        MRIVAA      *NONE      MRIVA
```

```
Opz  Definiz. commit   N° lav.   ID LUW                                   Stato
     *JOB              460667    APPN.[illegible].X'BCFEBB5E077B'.00001    RIPRISTINO
```

```
                    Visualizzazione stato definizione commit
                                                    20/01/26  12:45:29
Lavoro:   [illegible]A        Utente:   [illegible]        Numero:   460667

Sottopr.  . . . . . . . . . . . . :   *NONE
ID LUW (Logical Unit of Work) . . :   APPN.[illegible].X'BCFEBB5E077B'.00001
ID spazio blocco  . . . . . . . . :   UDB_01000000000409ED
Definizione commit  . . . . . . . :   *JOB
Gruppo attivazione  . . . . . . . :
Gruppo ASP  . . . . . . . . . . . :   *SYSBAS

Ubicazione risorsa  . . . . . . . :   NESSUNO
Livello blocco predefinito  . . . :   *CHG
Utente  . . . . . . . . . . . . . :   [illegible]
Modifiche in sospeso locali . . . :   NO
Ruolo . . . . . . . . . . . . . . :
Stato . . . . . . . . . . . . . . :   RIPRISTINO
  Registrazione data/ora  . . . . :

Risincronizz. in corso  . . . . . :   NO
Operaz. euristica . . . . . . . . :
Lavoro attivo . . . . . . . . . . :   SI
Lavoro server . . . . . . . . . . :
Ambito blocco . . . . . . . . . . :   *JOB
Supero tempo transazione  . . . . :
Attesa massima blocco . . . . . . :
```

21

# Demo - CALL PGMC/2

WRKCMTDFN

PGMC

```
DSPLY  Eseguito update 000200. Proseguire? (S/N)
```

```
Opz  Definiz. commit   N° lav.   Risinc. in corso   Modif. locali sospese
     *JOB              460667    NO                 SI
```

```
                    Visualizz. stato livello record
                                                    Sistema:   [illegible]
Lavoro:   [illegible]A        Utente:   [illegible]        Numero:   460667

Definizione commit . . . . . . . . . . :   *JOB

                              ----------Modifiche----------
File        Libreria    Membro      Commit   Rollback   Sospeso
EMPLOYEE    MK1SAMPLE   EMPLOYEE         0          0         1
```

```
File        Libreria    Membro      Ciclo commit
EMPLOYEE    MK1SAMPLE   EMPLOYEE    6727
```

```
                    Visualizzazione stato definizione commit
                                                    20/01/26  12:48:19
Lavoro:   [illegible]A        Utente:   [illegible]        Numero:   460667

Sottopr.  . . . . . . . . . . . . :   *NONE
ID LUW (Logical Unit of Work) . . :   APPN.[illegible].X'BCFEBB5E077B'.00001
ID spazio blocco  . . . . . . . . :   UDB_01000000000409ED
Definizione commit  . . . . . . . :   *JOB
Gruppo attivazione  . . . . . . . :
Gruppo ASP  . . . . . . . . . . . :   *SYSBAS

Ubicazione risorsa  . . . . . . . :   LOCALE
Livello blocco predefinito  . . . :   *CHG
Utente  . . . . . . . . . . . . . :   [illegible]
Modifiche in sospeso locali . . . :   SI
```

```
                    Visualizzazione dello stato del giornale
                                                    Sistema:   [illegible]
Lavoro:   [illegible]A        Utente:   [illegible]        Numero:   460667

Definizione commit . . . . . . . . . . :   *JOB

Immettere le opzioni e premere Invio.
  5=Visualizzaz. voci ciclo di commit   6=Visualizzaz. tutte le voci
  7=Gestione attributi giornale

                          Id
                          ciclo                  Blocchi     Modifiche
Opz  Giornale   Libr.     commit                 di record   in sospeso
     QSQJRN     MK1SAMPLE 6727                   1           1
```

```
                    Visualizzazione dettagli voce di giornale
Giornale . . . . . . :   QSQJRN         Libreria . . . . . . :   MK1SAMPLE

Sequenza . . . . . . :   6729
Codice . . . . . . . :   R  - Operazione su un record specifico
Tipo . . . . . . . . :   UP - Aggiornamento, immagine-successiva

Oggetto  . . . . . . :   EMPLOYEE
  Libreria . . . . . :     MK1SAMPLE
  Membro . . . . . . :     EMPLOYEE
  Tipo . . . . . . . :     *QDDS
Data . . . . . . . . :   20/01/26
Ora  . . . . . . . . :   12:46:46,837008
Contrassegno . . . . :   0
Conteggio/RRN  . . . :   18
ID ciclo di commit . :   6727
Livello di commit
  nidificata . . . . :   0
Lavoro . . . . . . . :   460667/[illegible]A
```

```
                    Visualizzazione delle voci di giornale
le . . . . . . . :   QSQJRN          Libreria . . . . . . :   [illegible]
 di sequenza più grande su questa schermata . : 0000000000000[illegible]
ere le opzioni e premere Invio.
sualizzazione intera voce

Sequenza  Cod.  Tipo  Oggetto     Libreria    Lavoro      Ora
    6727  C     SC    start ciclo di commit   [illegible]A  12[illegible]
    6728  R     UB    EMPLOYEE    MK1SAMPLE   [illegible]A  12[illegible]
    6729  R     UP    EMPLOYEE    MK1SAMPLE   [illegible]A  12[illegible]
```

22

# Demo - CALL PGMC/3

PGMC

```
DSPLY  Avvio programma PGMD. Invio per proseguire.
```

WRKCMTDFN

```
     Definiz.    N°      Uten.      Nome
Opz  commit      lav.    lav.       lav.       Sottopr.  Utente
     *JOB        460667  MRIVA      MRIVAA     *NONE     MRIVA
```

```
                    Visualizzazione stack di chiamata
                                                                  S
Lavoro:      [illegible]A        Utente:      [illegible]          Numero:
Sottoproc.:   0000016A

                                      -----Gruppo attivazione-----
Tipo  Programma                       Nome        Numero
      QCMD        QSYS                *DFTACTGRP  0000000000000001
      QUICMENU    QSYS                *DFTACTGRP  0000000000000001
   1  QUIMNDRV    QSYS                *DFTACTGRP  0000000000000001
   2  QUIMGFLW    QSYS                *DFTACTGRP  0000000000000001
   3  QUICMD      QSYS                *DFTACTGRP  0000000000000001
      PGMC        MK1SQL              *DFTACTGRP  0000000000000002
      PGMC        MK1SQL              *DFTACTGRP  0000000000000002
      PGMD        MK1SQL              *DFTACTGRP  0000000000000002
      PGMD        MK1SQL              *DFTACTGRP  0000000000000002
```

23

# Demo - CALL PGMC/4

PGMC

```
DSPLY  Eseguito update 000210. Proseguire? (S/N)
```

WRKCMTDFN

```
                    Visualizz. stato livello record
                                                        Sistema:   [illegible]
Lavoro:      [illegible]A        Utente:      [illegible]        Numero:   460667

Definizione commit . . . . . . . . . . :   *JOB

                                   ------------Modifiche------------
File        Libreria    Membro        Commit    Rollback    Sospeso
EMPLOYEE    MK1SAMPLE   EMPLOYEE           0           0          1
                        EMPLOYEE           0           0          1
```

```
                                          Id
                                          ciclo
File        Libreria    Membro            commit
EMPLOYEE    MK1SAMPLE   EMPLOYEE          6727
                        EMPLOYEE          6727
```

```
                          Id
                          ciclo                  Blocchi     Modifiche
Opz  Giornale    Libr.    commit                 di record   in sospeso
     QSQJRN      MK1SAMPLE 6727                  2           2
```

24

# Demo - CALL PGMC/5

WRKCMTDFN

PGMA

```
DSPLY  Avvio programma PGMA. Invio per proseguire.
```

```
                    Visualizzazione stack di chiamata
 Lavoro:    [redacted]A        Utente:   [redacted]        Numero:
 Sottoproc.:   0000016A

                                  -----Gruppo attivazione-----
 Tipo  Programma                  Nome        Numero
       QCMD       QSYS            *DFTACTGRP  0000000000000001
       QUICMENU   QSYS            *DFTACTGRP  0000000000000001
    1  QUIMNDRV   QSYS            *DFTACTGRP  0000000000000001
    2  QUIMGFLW   QSYS            *DFTACTGRP  0000000000000001
    3  QUICMD     QSYS            *DFTACTGRP  0000000000000001
       PGMC       MK1SQL          *DFTACTGRP  0000000000000002
       PGMC       MK1SQL          *DFTACTGRP  0000000000000002
       PGMD       MK1SQL          *DFTACTGRP  0000000000000002
       PGMD       MK1SQL          *DFTACTGRP  0000000000000002
       PGMA       MK1SQL          *NEW        0000000000000013
       PGMA       MK1SQL          *NEW        0000000000000013
```

```
     Definiz.   N°      Uten.      Nome
Opz  commit     lav.    lav.       lav.       Sottopr.  Utente
     *JOB       460667  MRIVA      MRIVAA     *NONE     MRIVA
```

```
                                   ------------Modifiche-------------
File        Libreria    Membro        Commit    Rollback    Sospeso
EMPLOYEE    MK1SAMPLE   EMPLOYEE           0           0          1
                        EMPLOYEE           0           0          1
                        EMPLOYEE           0           0          1
```

```
                                                   Id
                                                   ciclo
File        Libreria    Membro                     commit
EMPLOYEE    MK1SAMPLE   EMPLOYEE                   6727
                        EMPLOYEE                   6727
                        EMPLOYEE                   6727
```

```
                    Visualizzazione gruppo attivazione
                                                 Sistema:  [redacted]
 Lavoro:    [redacted]A        Utente:   [redacted]        Numero:   460667

 -----Gruppo attivazione------
 Nome        Numero            Indicatore in uso   Programma   Libreria
 *DFTACTGRP  0000000000000001      Si
 *DFTACTGRP  0000000000000002      Si
 QLGLOCAL    0000000000000011      Senza           QLGLOCAL    QSYS
 QSQCLI      0000000000000012      Senza           QSQCLI      QSYS
 19          0000000000000013      Si              PGMA        MK1SQL
```



# Demo - CALL PGMC/6

WRKCMTDFN

PGMB

```
DSPLY  Avviato contr.sincr. *ACTGRP. Invio per proseguire
```

```
                    Visualizzazione stack di chiamata
 Lavoro:    [redacted]A        Utente:   [redacted]        Numero:
 Sottoproc.:   0000016A

                                  -----Gruppo attivazione-----
 Tipo  Programma                  Nome        Numero
       QCMD       QSYS            *DFTACTGRP  0000000000000001
       QUICMENU   QSYS            *DFTACTGRP  0000000000000001
    1  QUIMNDRV   QSYS            *DFTACTGRP  0000000000000001
    2  QUIMGFLW   QSYS            *DFTACTGRP  0000000000000001
    3  QUICMD     QSYS            *DFTACTGRP  0000000000000001
       PGMC       MK1SQL          *DFTACTGRP  0000000000000002
       PGMC       MK1SQL          *DFTACTGRP  0000000000000002
       PGMB       MK1SQL          *NEW        0000000000000014
       PGMB       MK1SQL          *NEW        0000000000000014
```

```
     Definiz.   N°      Uten.      Nome
Opz  commit     lav.    lav.       lav.       Sottopr.  Utente
     *JOB       460667  [redacted]  [redacted]A  *NONE     [redacted]
     20         460667  [redacted]  [redacted]A  *NONE     [redacted]
```

```
                    Visualizzazione stato definizione commit
                                                 [redacted]
                                                 20/01/26  15:22:52
 Lavoro:    [redacted]A        Utente:   [redacted]        Numero:   460667

 Sottopr.  . . . . . . . . . . . . :   *NONE
 ID LUW (Logical Unit of Work) . . :   APPN.[redacted].X'BD21C5AF077B'.00001
 ID spazio blocco  . . . . . . . . :   UDB_01000000000409F5
 Definizione commit  . . . . . . . :   20
 Gruppo attivazione  . . . . . . . :   20
 Gruppo ASP  . . . . . . . . . . . :   *SYSBAS

 Ubicazione risorsa  . . . . . . . :   NESSUNO
 Livello blocco predefinito  . . . :   *CHG
 Utente  . . . . . . . . . . . . . :   MRIVA
 Modifiche in sospeso locali . . . :   NO
 Ruolo . . . . . . . . . . . . . . :
 Stato . . . . . . . . . . . . . . :   RIPRISTINO
```

```
                    Visualizzazione gruppo attivazione
                                                 Sistema:  [redacted]
 Lavoro:    [redacted]A        Utente:   [redacted]        Numero:   460667

 -----Gruppo attivazione------
 Nome        Numero            Indicatore in uso   Programma   Libreria
 *DFTACTGRP  0000000000000001      Si
 *DFTACTGRP  0000000000000002      Si
 QLGLOCAL    0000000000000011      Senza           QLGLOCAL    QSYS
 QSQCLI      0000000000000012      Senza           QSQCLI      QSYS
 20          0000000000000014      Si              PGMB        MK1SQL
```

```
Risincronizz. in corso  . . . . . :   NO
Operaz. euristica . . . . . . . . :
Lavoro attivo . . . . . . . . . . :   SI
Lavoro server . . . . . . . . . . :
Ambito blocco . . . . . . . . . . :   *ACTGRP
Supero tempo transazione  . . . . :
Attesa massima blocco . . . . . . :
```

# Demo - CALL PGMC/7

WRKCMTDFN

PGMB

```
DSPLY  Eseguito update 000240. Proseguire? (S/N)
```

```
                                  Risinc.   Modif.
     Definiz.     N°              in        locali
Opz  commit       lav.            corso     sospese
     *JOB         460667          NO        SI
     20           460667          NO        SI
```

```
                    Visualizz. stato livello record
                                                        Sistema:
Lavoro:                Utente:                  Numero:   460667

Definizione commit . . . . . . . . . . :   20

                                  ------------Modifiche------------
File        Libreria    Membro          Commit    Rollback     Sospeso
EMPLOYEE    MK1SAMPLE   EMPLOYEE             0           0           1
```

```
Definizione commit . . . . . . . . . . :   20

                                                  Id
                                                  ciclo
File        Libreria    Membro                    commit
EMPLOYEE    MK1SAMPLE   EMPLOYEE                  6735
```

```
                                  Id
                                  ciclo                   Blocchi     Modifiche
Opz  Giornale     Libr.           commit                  di record   in sospeso
     QSQJRN       MK1SAMPLE       6735                    1           1
```

```
Sequenza  Cod.  Tipo  Oggetto     Libreria     Lavoro     Ora
    6735   C     SC   start ciclo di commit
    6736   R     UB   EMPLOYEE    MK1SAMPLE    [redacted]  15:25:09
    6737   R     UP   EMPLOYEE    MK1SAMPLE    [redacted]  15:25:09
```

27

# Demo - CALL PGMC/8

WRKCMTDFN

PGMB

```
DSPLY  PGMB ha eseguito commit. Invio per proseguire
```

```
                                  Risinc.   Modif.
     Definiz.     N°              in        locali
Opz  commit       lav.            corso     sospese
     *JOB         460667          NO        SI
     20           460667          NO        NO
```

```
                                  ------------Modifiche------------
File        Libreria    Membro          Commit    Rollback     Sospeso
EMPLOYEE    MK1SAMPLE   EMPLOYEE             1           0           0
```

```
                    Visualizzazione stato definizione commit
                                                        20/01/26  15:31:25
Lavoro:                Utente:                  Numero:   460667

Sottopr.  . . . . . . . . . . . . . :   *NONE
ID LUW (Logical Unit of Work) . . . :   APPN.[redacted].X'BD21C5AF077B'.00002
ID spazio blocco  . . . . . . . . . :   UDB_01000000000409F5
Definizione commit  . . . . . . . . :   20
Gruppo attivazione  . . . . . . . . :   20
Gruppo ASP  . . . . . . . . . . . . :   *SYSBAS

Limite blocco . . . . . . . . . . . :   500000000
Commit durevole . . . . . . . . . . :   SI
Numero di commit  . . . . . . . . . :   1
Numero di rollback  . . . . . . . . :   0
Giornale predefinito  . . . . . . . :
  Libreria  . . . . . . . . . . . . :
```

28

# Demo - CALL PGMC/9

WRKCMTDFN

PGMB

```
DSPLY  PGMB arresterà il contr.sincr. Invio per proseguire
DSPLY  Arresto controllo di sincronia con errori
ENDCMTCTL non è consentito. Sono presenti delle modifiche in sospeso.
```

```
                    Visualizz. stato livello record
                                                          Sistema:
 Lavoro:          Utente:                  Numero:   460667

 Definizione commit . . . . . . . . . . :   20

                                   Liv                          Risoluzione
 File        Libreria    Membro   blocco  Stato              accessi contemp.
 EMPLOYEE    MK1SAMPLE   EMPLOYEE  *CHG   APERTO                  *WAIT
```

```
DSPLY  PGMB ha chiuso il file EMPLOYEE. Invio per prosegui
```

```
                    Visualizz. stato livello record
                                                          Sistema:   AS1PRD
 Lavoro:          Utente:                  Numero:   460667

 Definizione commit . . . . . . . . . . :   20

                                   Liv                          Risoluzione
 File        Libreria    Membro   blocco  Stato              accessi contemp.

   (Non ci sono modifiche a livello record sotto controllo di commit.)
```

```
DSPLY  PGMB arresterà il contr.sincr. Invio per proseguire
```

```
     Definiz.    N°      Uten.      Nome
Opz  commit     lav.    lav.       lav.        Sottopr.  Utente
     *JOB       460667                          *NONE
```

29

# Demo - CALL PGMC/10

WRKCMTDFN

PGMC

```
DSPLY  PGMC arresterà il contr.sincr. Invio per proseguire
DSPLY  Arresto controllo di sincronia con errori
```

```
                    Visualizz. stato livello record
                                                          Sistema:
 Lavoro:          Utente:                  Numero:   460667

 Definizione commit . . . . . . . . . . :   *JOB

                                   Liv                          Risoluzione
 File        Libreria    Membro   blocco  Stato              accessi contemp.
 EMPLOYEE    MK1SAMPLE   EMPLOYEE  *CHG   APERTO                  *WAIT
                         EMPLOYEE         CHIUSO
```

Chiusura PGMC

```
     Definiz.    N°      Uten.      Nome
Opz  commit     lav.    lav.       lav.        Sottopr.  Utente
     *JOB       460667  MRIVA      MRIVAA      *NONE     MRIVA
```

SIGNOFF

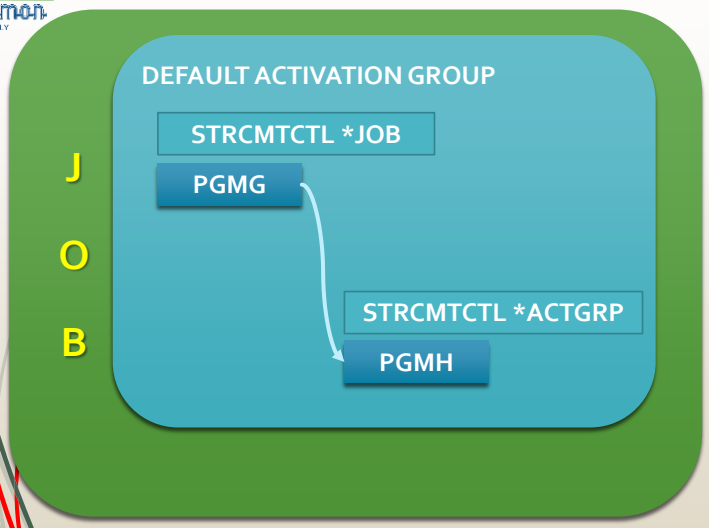
```
     Definiz.    N°      Uten.      Nome
Opz  commit     lav.    lav.       lav.        Sottopr.  Utente

  (Non ci sono definizioni di commit attive)
```

```
CPF8356    Diagnostica            30   20/01/26  15:41:04,336486  QTNEND      QSYS       2075     QWTPITP2    QSYS        0439
                                    Messaggio . . . :   Il controllo di commit è terminato con 3 modifiche non
                                      sottoposte a commit.
 5770SS1 V7R5M0 220415                Registrazione lavoro                 AS1PRD    20/01/26  15:41:04 CET     Pag.   14
  Nome lavoro  . . . . . . . . . :   MRIVAA        Utente  . . . . . . . : MRIVA        Numero . . . . . . . . . . . . :   460667
  Descrizione lavoro . . . . . . :   QDFTJOBD       Libreria  . . . . :  QGPL
IDMSG      TIPO                    GRAV  DATA      ORA            DA PGM     LIBRERIA     INST     A PGM      LIBRERIA     INST
                                      Causa . . :  È stato eseguito il rollback di modifiche in sospeso poiché vi
                                      erano modifiche non sottoposte a commit nel momento in cui è terminato il
                                      controllo di commit per la definizione di commit *JOB con un id LUW (logical
                                      unit of work/unità logica di lavoro) APPN.AS1PRD.X'BCFEBB5E077B'.00001.
```

30

## Comm.It. quiz



PGMH può aprire una definizione di controllo sincronia a livello *ACTGRP?

YES NO

PGMG.RPGLE, PGMH.RPGLE

31

## Controllo sincronia - isolamento transazioni/3

Solo programmi in ambiente ILE possono aprire transazioni in un gruppo di attivazione diverso da quello di default

in un lavoro **possono esistere più transazioni contemporanee** e **indipendenti** *se e solo se* nel job sono in esecuzione uno o più programmi ILE

Ogni transazione può allocare al max 500.000.000 di record, ma si consiglia *(IBM documentation)* di non superare i 2.000 record per transazione

32

## Controllo sincronia - isolamento transazioni/4

In un **ambiente misto OPM e ILE** se si desidera che tutti i programmi **condividano la medesima definizione** di controllo di sincronia è essenziale che l'ambito sia ***JOB**

33

## Livelli di isolamento delle transazioni

integrità e sicurezza del dato

| Livello | Descrizione | Attivo | Letture sporche | Record fantasma | Allocazione record letti | Note |
|---|---|---|---|---|---|---|
| *RR/RR | **LETTURA RIPETIBILE** (Repeatable Read or Serializable) | ✓ | ✗ | ✗ | ✓ | Garantisce che i record letti non possono essere modificati da un altro gruppo di attivazione e che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato<br>Alloca tutti i record letti e il gruppo di attivazione è completamente isolato rispetto ad altri |
| *ALL/RS | **LETTURA STABILE** (Read Stability) | ✓ | ✗ | ✓ | ✓ | *Garantisce che i record letti non possono essere modificati da un altro gruppo di attivazione e che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato → come RR*<br>Record "fantasma": a differenza di RR eseguendo più volte la stessa query possono comparire i record aggiunti in altri gruppi di attivazione |
| *CS/CS | **LETTURA SINCRONIZZATA** (Cursor Stability or Read Committed) | ✓ | ✗ | ✓ | ✗ | *Garantisce che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato → come RR o RS*<br>A differenza di RR e RS i record letti possono essere modificati da altri gruppi di attivazione |
| *CHG/UR | **LETTURA NON SINCRONIZZATA** (Uncommitted Read) | ✓ | ✓ | ✓ | ✗ | Livello più basso di controllo di sincronia.<br>Consente di leggere record modificati in altri gruppi di attivazione anche se non ancora consolidate |
| *NONE/NC | **NESSUNO** | ✗ | ✓ | ✓ | ✗ | Ogni singolo aggiornamento è effettivamente sincronizzato quando viene completato. Non è possibile eseguire commit/rollback |

https://www.ibm.com/docs/en/i/7.6.0?topic=concepts-isolation-level

34

## Livelli di isolamento delle transazioni/2

- ***Letture sporche***: è possibile leggere dati che sono stati variati ma non sincronizzati da un altro lavoro.
- ***Record fantasma***: la LUW 1 legge un set di record che soddisfa certi criteri. La LUW 2 inserisce un nuovo record che soddisfa i criteri di selezione della query della LUW 1. La LUW 1 riesegue la query e legge anche I nuovi record
- ***Lettura non ripetibile***: la LUW 1 legge un record. La LUW 2 modifica quel record e lo consolida. La LUW1 rilegge il medesimo record ed ottiene i nuovi dati consolidati dalla LUW 2

https://www.ibm.com/docs/en/i/7.6.0?topic=level-comparison-isolation-levels

35

## Livello di allocazione

- Il parametro LCKLVL in STRCMTCTL determina il livello di allocazione dei record di default

integrità e sicurezza del dato

| Tipo | Descrizione | Conseguenze |
|---|---|---|
| *ALL | allocazione dei record modificati e letti durante tutta la transazione | - anche un record letto senza scopo di aggiornamento è allocato |
| *CS | allocazione dei record modificati durante tutta la transazione<br>allocazione dei record letti fino al rilascio o alla successiva lettura | - un altro lavoro non può leggere record per aggiornamento che sono già stati letti dal lavoro corrente<br>- il lavoro corrente non può leggere record per aggiornamento che sono stati allocati di tipo *update in un altro lavoro |
| *CHG | allocazione dei record modificati durante tutta la transazione | |

https://www.ibm.com/docs/en/i/7.6.0?topic=control-commit-lock-level

36

# Prerequisiti per attivare controllo di sincronia

Specifiche di controllo e definizione file

Opzioni precompilatore SQL

37

## H specifiche di controllo

- Keyword per activation group
  - `DFTACTGRP(*YES|*NO)`
  - `ACTGRP(*STGMDL|*NEW|*CALLER|'nome')`

## F specifiche definizione file

- keyword `COMMIT`
- keyword `USROPN` `COMMIT(ind-variable)`
  se la variabile *`ind-variable`* è impostata a `'1'` il controllo di sincronia è attivato, se `'0'` invece non viene attivato

38

## STRCMTCTL / ENDCMTCTL

- STRCMTCTL
  **avvia** una definizione di controllo di sincronia
  - Se già attivo → errore CPF8351
- ENDCMTCTL
  **arresta** una definizione di controllo di sincronia
  - Interattivo: se eseguito con modifiche in sospeso o con risorse aperte → CPA8350 o CPF8355
    - CM: esegue commit e prosegue
    - RB: esegue rollback e prosegue
  - Batch: se eseguito con modifiche in sospeso o con risorse aperte → rollback

39

## Embedded SQL

- Parametro COMMIT nei comandi di compilazione
- Oppure nel sorgente si può specificare la direttiva di compilazione con l'istruzione SQL set option. P.es.:
  ```
  exec sql set option commit = *CHG;
  ```
- Oppure il livello di isolamento può essere specificato in ogni singola istruzione SQL DELETE, INSERT, SELECT, UPDATE (*isolation-clause*)
  ```
  ... with ur;
  ```
- Implicitamente viene avviato il controllo di sincronia a livello di **activation group**

https://www.ibm.com/docs/en/i/7.6.0?topic=statement-isolation-clause

40

## Chiudere una transazione/1

- Una transazione viene chiusa esplicitamente con le operazioni di commit (vengono consolidate le modifiche) o rollback (vengono annullate le modifiche)
- I codici operativi RPG per commit e rollback sono: COMMIT, ROLBK
- Le istruzioni SQL per commit e rollback sono

```
exec sql commit;
exec sql rollback;
```

41

## Chiudere una transazione/2

- **COMMIT**
  - Consolida tutte le modifiche eseguite sui record dal precedente commit/rollback
  - Rilascia tutti i lock sui record
  - Non viene alterata la posizione dei file
- **ROLLBACK**
  - Annulla tutte le modifiche eseguite sui record a partire dal precedente commit/rollback o dal savepoint (se embedded SQL)
  - Rilascia tutti i lock sui record
  - Riposiziona i file alla posizione *al momento del precedente commit* (cfr. esempio slide 60)

42

# Commit e rollback

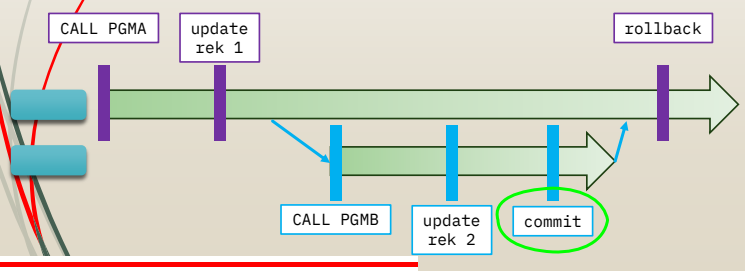
Consolidare o annullare una transazione

43

## Comm.lt. quiz

PGMA e PGMB vengono eseguiti entrambi nel *DFTACTGRP con COMMIT = *CHG

- PGMA esegue un update del record 1, quindi chiama il PGMB
- PGMB esegue un update del record 2, esegue commit e ritorna al chiamante
- PGMA esegue rollback



Quanti record vengono consolidati?

0 1 2

44

PGMA -> PGMB
*dft*

## Esempio transazioni/1

- PGMA e PGMB vengono eseguiti entrambi nel *DFTACTGRP con COMMIT = *CHG

PGMA esegue un update, quindi chiama il PGMB
PGMB esegue un update di un altro record, esegue commit e ritorna al chiamante
PGMA esegue rollback

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'48953C039E92'.00001 | 1 ✓ |
| PGMB | update 1 record | *DFTACTGRP | APPN.*sysname*.X'48953C039E92'.00001 | 2 ✓ |
| PGMB | commit | *DFTACTGRP | APPN.*sysname*.X'48953C039E92'.00002 | 0 |
| PGMB | return | | | |
| PGMA | rollback | *DFTACTGRP | APPN.*sysname*.X'48953C039E92'.00003 | 0 |

**RISULTATO: <u>entrambi</u> gli update vengono consolidati sul db dal commit eseguito da PGMB**

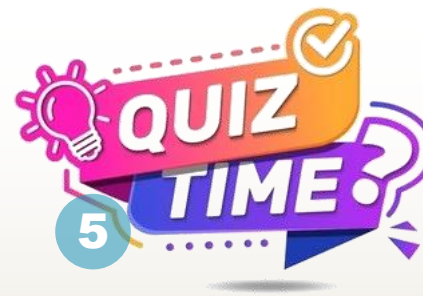
✳ TCMT1A.SQLRPGLE, TCMT1B.SQLRPGLE

45

## Comm.lt. quiz

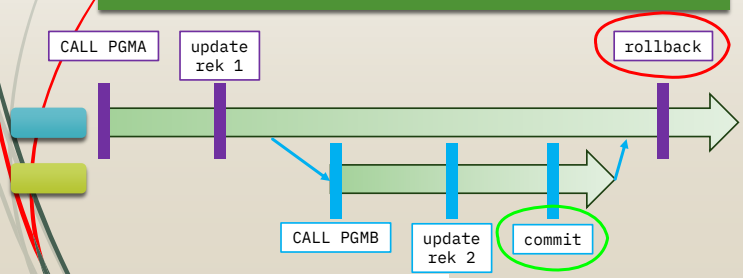
PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG

- PGMA esegue un update del record 1, quindi chiama il PGMB
- PGMB esegue un update del record 2, esegue commit e ritorna al chiamante
- PGMA esegue rollback




Quanti record vengono consolidati?

0 1 2




46

PGMA *dft* | PGMB *new*

## Esempio transazioni/2

- PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, esegue commit e ritorna al chiamante
  PGMA esegue rollback

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00001 | 1 ✗ |
| PGMB | update 1 record | act group 19 | APPN.*sysname*.X'5A75E299C288'.00001 | 1 ✓ |
| PGMB | commit | act group 19 | APPN.*sysname*.X'5A75E299C288'.00002 | 0 |
| PGMB | return | *si chiude automaticamente act group 19 perché gestito dal sistema* | | |
| PGMA | rollback | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00002 | 0 |

**RISULTATO: solo l'update eseguito da PGMB viene consolidato sul db**

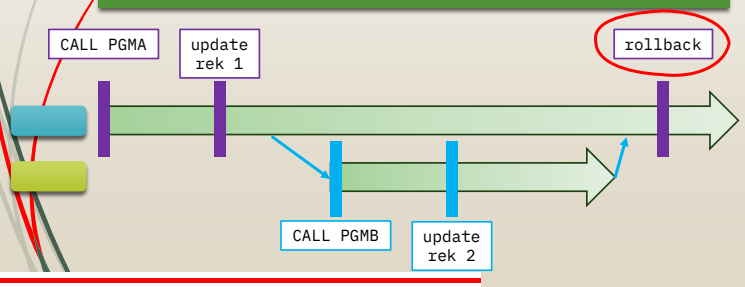
TCMT2A.SQLRPGLE, TCMT2B.SQLRPGLE

47

## Comm.lt. quiz

PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG

- PGMA esegue un update del record 1, quindi chiama il PGMB
- PGMB esegue un update del record 2 e ritorna al chiamante
- PGMA esegue rollback



Quanti record vengono consolidati?

0 1 2

Chi ha consolidato il record n. 2?

48

PGMA *dft* | PGMB *new*

## Esempio transazioni/3

- PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, NON esegue commit esplicito e ritorna al chiamante
  PGMA esegue rollback

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00001 | 1 ✗ |
| PGMB | update 1 record | act group 19 | APPN.*sysname*.X'5A75E299C288'.00001 | 1 ✓ |
| PGMB | return | *si chiude act group 19 e viene eseguito un **commit implicito*** | | |
| PGMA | rollback | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00002 | 0 |

**RISULTATO: solo l'update eseguito da PGMB viene consolidato sul db**

TCMT3A.SQLRPGLE, TCMT3B.SQLRPGLE

49

PGMA *dft* | PGMB *name*

## Esempio transazioni/4

- PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP('PGMB') con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, esegue commit e ritorna al chiamante
  PGMA esegue rollback

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00001 | 1 ✗ |
| PGMB | update 1 record | act group PGMB | APPN.*sysname*.X'5A75E299C288'.00001 | 1 ⌛ |
| PGMB | return | *non si chiude act group PGMB perché non gestito dal sistema e la transazione rimane sospesa* | | |
| PGMA | rollback | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00002 | 0 |

**RISULTATO: l'update eseguito da PGMB rimane in sospeso fino a quando non viene chiuso l'actgrp PGMB**

TCMT4A.SQLRPGLE, TCMT4B.SQLRPGLE

50

PGMA *dft* | PGMB *new*

## Esempio transazioni/5

- PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB con estensore errore CALLP(E)
  PGMB esegue un update di un altro record e prima di eseguire commit si interrompe per un'eccezione non prevista, quindi ritorna a PGMA che prosegue all'istruzione successiva
  PGMA esegue rollback

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00001 | 1 × |
| PGMB | update 1 record | act group 19 | APPN.*sysname*.X'5A75E299C288'.00001 | 1 × |
| PGMB | eccezione | *si chiude act group 19 e viene eseguito un **rollback implicito*** | | |
| PGMA | rollback | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00002 | 0 |

**RISULTATO: nessun update viene consolidato sul db**

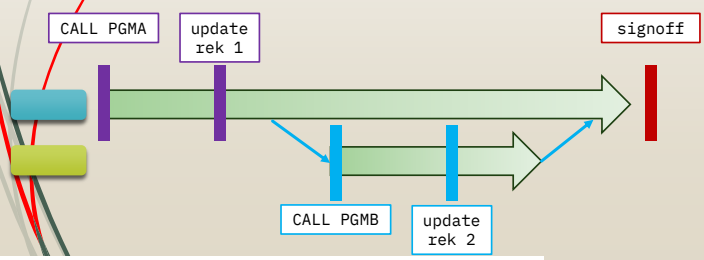
TCMT5A.SQLRPGLE, TCMT5B.SQLRPGLE

51

## Comm.It. quiz

PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG

- PGMA esegue un update del record 1, quindi chiama il PGMB
- PGMB esegue un update del record 2 e ritorna al chiamante
- PGMA si chiude normalmente
- viene chiuso normalmente il job (signoff)




Quanti record vengono consolidati?

0 1 2

52

PGMA *dft* | PGMB *new*

## Esempio transazioni/6

- PGMA viene eseguito in *DFTACTGRP e PGMB in ACTGRP(*NEW) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, NON esegue commit esplicito e ritorna al chiamante
  PGMA si chiude normalmente senza eseguire commit e poi viene chiuso normalmente il job

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | *DFTACTGRP | APPN.*sysname*.X'5A75B9A8C288'.00001 | 1 ✗ |
| PGMB | update 1 record | act group 19 | APPN.*sysname*.X'5A75E299C288'.00001 | 1 ✓ |
| PGMB | return | *si chiude act group 19 e viene eseguito un **commit implicito*** | | |
| PGMA | return | *transazione rimane sospesa con il rek aggiornato da PGMA ancora allocato* | | |
| == | endjob | *si chiude il job e viene eseguito un **rollback implicito*** | | |

**RISULTATO: solo l'update eseguito da PGMB viene consolidato sul db**

TCMT6A.SQLRPGLE, TCMT6B.SQLRPGLE

53

PGMA -> PGMB *new - caller*

## Esempio transazioni/7

- PGMA viene eseguito in ACTGRP(*NEW) e PGMB in ACTGRP(*CALLER) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, NON esegue commit esplicito e ritorna al chiamante
  PGMA esegue commit

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | act group 19 | APPN.*sysname*.X'48953C039E92'.00001 | 1 ✓ |
| PGMB | update 1 record | act group 19 | APPN.*sysname*.X'48953C039E92'.00001 | 1 ✓ |
| PGMB | return | *NON si chiude act group 19 e quindi non viene eseguita nessuna azione implicita* | | |
| PGMA | commit | act group 19 | APPN.*sysname*.X'48953C039E92'.00002 | 0 |
| PGMA | return | *si chiude act group 19 e viene eseguito un **commit implicito** anche se non eseguito esplicitamente da pgmA* | | |

**RISULTATO: <u>entrambi</u> gli update vengono consolidati sul db dal commit eseguito da PGMA**

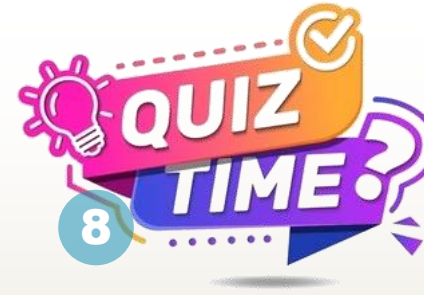
TCMT7A.SQLRPGLE, TCMT7B.SQLRPGLE

54

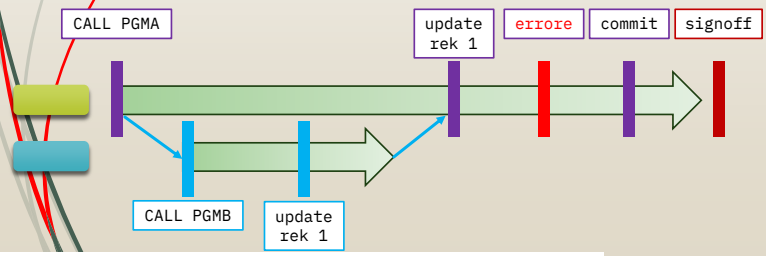
## Comm.It. quiz

PGMA viene eseguito in ACTGRP(*NEW) e PGMB in *DFTACTGRP con COMMIT = *CHG

- PGMA chiama il PGMB
- PGMB esegue update del record 1 e ritorna al chiamante
- PGMA tenta di eseguire update sullo stesso record modificato da PGMB




Quanti record vengono consolidati?




0 1

55

## Esempio transazioni/8

PGMA *new* | PGMB *dft*

PGMA viene eseguito in ACTGRP(*NEW) e PGMB in *DFTACTGRP con COMMIT = *CHG
PGMA chiama il PGMB; PGMB esegue un update di un record ma NON esegue commit esplicito e ritorna al chiamante; PGMA tenta di eseguire update sullo stesso record modificato da PGMB

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | *si apre act group* **22** | | APPN.*sysname*.X'5A779697C288'.00001 | |
| PGMB | update record *n* | *DFTACTGRP | APPN.*sysname*.X'5A77D18EC288'.00001 | 1 × |
| PGMB | return | *NON si chiude dftactgrp e non viene eseguita nessuna azione implicita transazione rimane sospesa con il rek aggiornato da PGMB allocato* | | |
| PGMA | update record *n* | act group 22 | ***errore record allocato*** | |
| PGMA | commit | act group 22 | APPN.*sysname*.X'5A779697C288'.00002 | |
| PGMA | return | si chiude act group 22 *ma la transazione del *dftactgrp rimane sospesa con il rek aggiornato da PGMB ancora allocato* | | |
| == | endjob | *si chiude il job e viene eseguito un* ***rollback implicito*** | | |

**RISULTATO: l'update di PGMA fallisce, l'update di PGMB rimane in sospeso fino a che si chiude il job**

TCMT8A.SQLRPGLE, TCMT8B.SQLRPGLE

56

PGMA *new* | PGMB *new*

# Esempio transazioni/9

- PGMA e PGMB vengono eseguiti in due ACTGRP(*NEW) con COMMIT = *CHG
  PGMA esegue un update, quindi chiama il PGMB
  PGMB esegue un update di un altro record, esegue commit e ritorna al chiamante
  PGMA esegue commit

| Programma | Azione | Ambito ctl sincronia | ID LUW | Modifiche in sospeso |
|---|---|---|---|---|
| PGMA | update 1 record | act group 21 | APPN.*sysname*.X'5A6EF3A9C263'.00001 | 1 ✓ |
| PGMB | update 1 record | act group 22 | APPN.*sysname*.X'5A6FE4BBC263'.00001 | 1 ✓ |
| PGMB | commit | act group 22 | APPN.*sysname*.X'5A6FE4BBC263'.00002 | 0 |
| PGMA | commit | act group 21 | APPN.*sysname*.X'5A6EF3A9C263'.00002 | 0 |

**RISULTATO: ogni update eseguito da PGMA e PGMB viene consolidato sul db dal proprio commit in modo indipendente**

✳ TCMT9A.SQLRPGLE, TCMT9B.SQLRPGLE



# Commit o rollback implicito

| Contesto | Azione implicita |
|---|---|
| Programma si interrompe con eccezione non prevista | rollback |
| Gruppo di attivazione si chiude normalmente | commit |
| Gruppo di attivazione si chiude con errori | rollback |
| Definizioni di commit in ambito *JOB o *DFTACTGRP | nessuna |
| Chiusura di un lavoro *JOB normale o anomala | rollback |

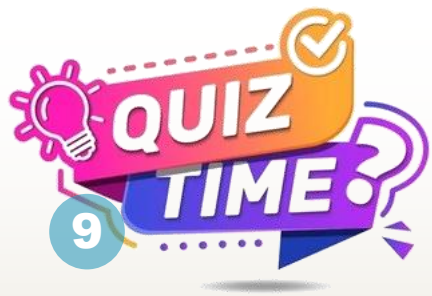
https://www.ibm.com/docs/en/i/7.6.0?topic=control-system-initiated-end-commitment

# Concorrenza di accesso

Livello di isolamento delle transazioni

59

## Comm.lt. quiz




*Cosa succede se tento di leggere i record modificati in una transazione in esecuzione su un altro job o activation group?*

Dipende dall'impostazione del livello di isolamento delle transazioni:

*CHG, *CS, *ALL, *RR

Il livello di isolamento di default è *CHG

60

# Concorrenza di accesso: esempi

Programma che apre una transazione per aggiornare dei record

```
dcl-s Domanda char(51);
dcl-s Risposta char(1);

exec sql
  set option COMMIT =  *CHG;
                       *CS;
                       *CS, CONACC = *CURCMT;
                       *RS;
                       *RS;

exec sql
  update EMPLOYEE
     set BONUS = BONUS + 10
     where EMPNO = '000010';

  Domanda = 'Confermi la modifica di EMPLOYEE? (S/N)';
  dsply Domanda ' ' Risposta;

  if %upper(Risposta) = 'S';
    exec sql
      commit;
  else;
    exec sql
      rollback;
  endif;

*inlr = *on;
return;
```

TCMTCTL1*.SQLRPGLE

Programma che legge lo stesso record durante la transazione

```
dcl-s Domanda char(51);
dcl-s Risposta char(1);
dcl-s wBonus packed(9:2);

exec sql
  set option COMMIT =  *CHG;
                       *CS;
                       *CS, CONACC = *CURCMT;
                       *RS;
                       *RS;

dou %upper(Risposta) <> 'S';
  exec sql
    select BONUS
      into :wBonus
      from EMPLOYEE
      where EMPNO = '000010';

  Domanda = 'BONUS di EMPNO 000010 è ' +
            %char(wBonus) + '. Rileggo? (S/N)';
  dsply Domanda ' ' Risposta;
enddo;

*inlr = *on;
return;
```

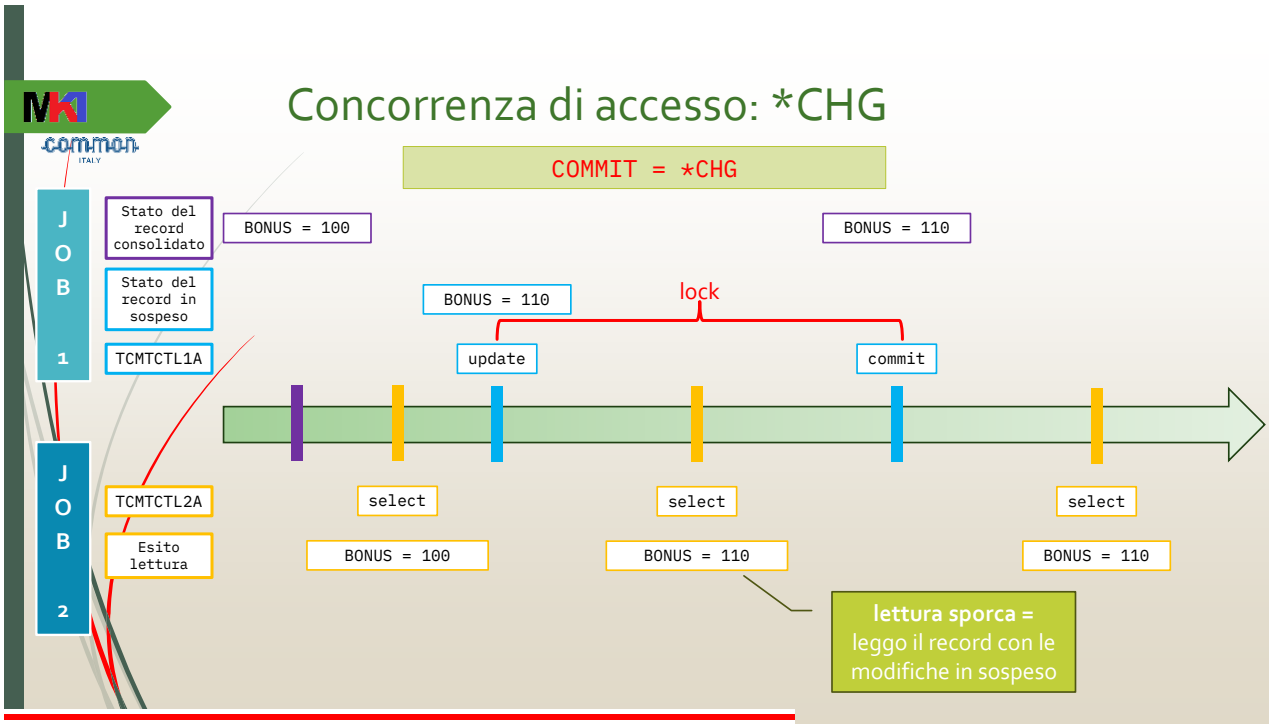
TCMTCTL2*.SQLRPGLE

61

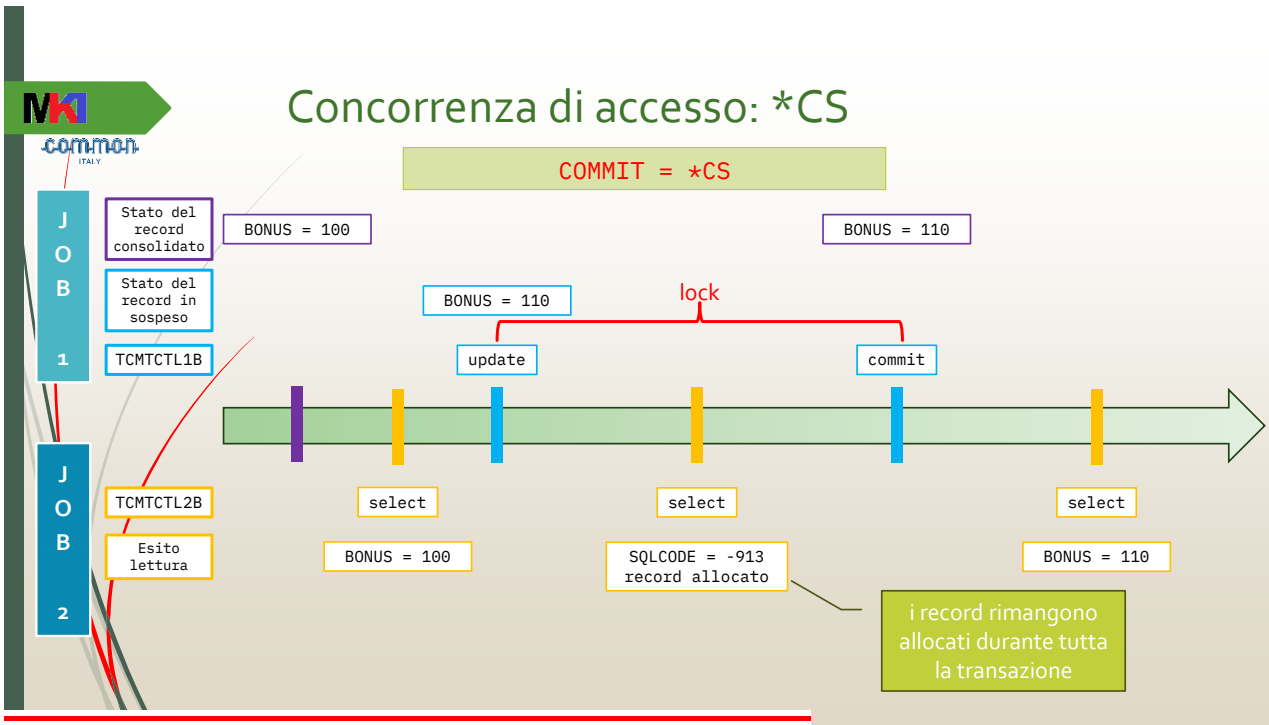
# *CHG vs *CS

| Livello | Descrizione | Attivo | Letture sporche | Record fantasma | Allocazione record letti | Note |
|---|---|---|---|---|---|---|
| *CS/CS | **LETTURA SINCRONIZZATA** (Cursor Stability or Read Committed) | ✓ | ✗ | ✓ | ✗ | *Garantisce che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato → come RR o RS* A differenza di RR e RS i record letti possono essere modificati da altri gruppi di attivazione |
| *CHG/UR | **LETTURA NON SINCRONIZZATA** (Uncommitted Read) | ✓ | ✓ | ✓ | ✗ | Livello più basso di controllo di sincronia. Consente di leggere record modificati in altri gruppi di attivazione anche se non ancora consolidate |

62


Concorrenza di accesso: *CHG
COMMIT = *CHG
JOB 1
Stato del record consolidato
BONUS = 100
BONUS = 110
Stato del record in sospeso
BONUS = 110
lock
TCMTCTL1A
update
commit
JOB 2
TCMTCTL2A
select
select
select
Esito lettura
BONUS = 100
BONUS = 110
BONUS = 110
lettura sporca = leggo il record con le modifiche in sospeso

63


Concorrenza di accesso: *CS
COMMIT = *CS
JOB 1
Stato del record consolidato
BONUS = 100
BONUS = 110
Stato del record in sospeso
BONUS = 110
lock
TCMTCTL1B
update
commit
JOB 2
TCMTCTL2B
select
select
select
Esito lettura
BONUS = 100
SQLCODE = -913 record allocato
BONUS = 110
i record rimangono allocati durante tutta la transazione

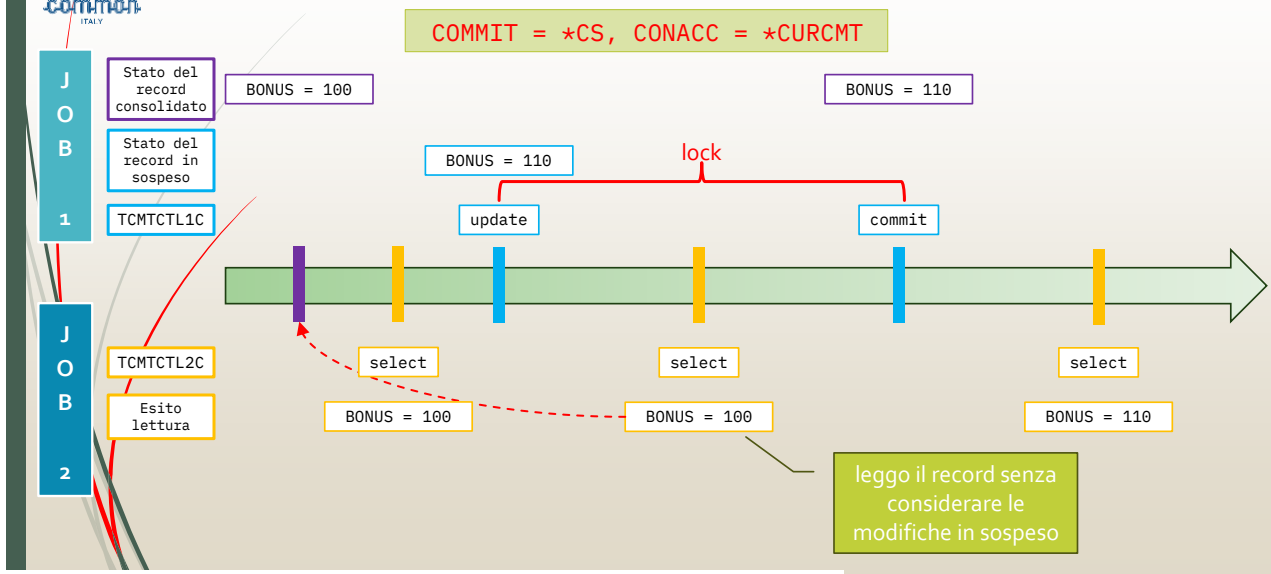
64

## Concurrent access resolution

- opzione CONACC
  - *CURCMT (currently committed): vengono letti i dati correntemente in stato consolidato ignorando le modifiche in sospeso
  - *WAIT: attende che la transazione che alloca il record venga chiusa con commit o rollback; se viene superato il tempo di attesa del record (WAITRCD) viene restituito sqlcode = -913
  - *DFT: non viene modificata l'impostazione e viene ereditata quella precedentemente impostata

65

## Concorrenza di accesso con *CURCMT



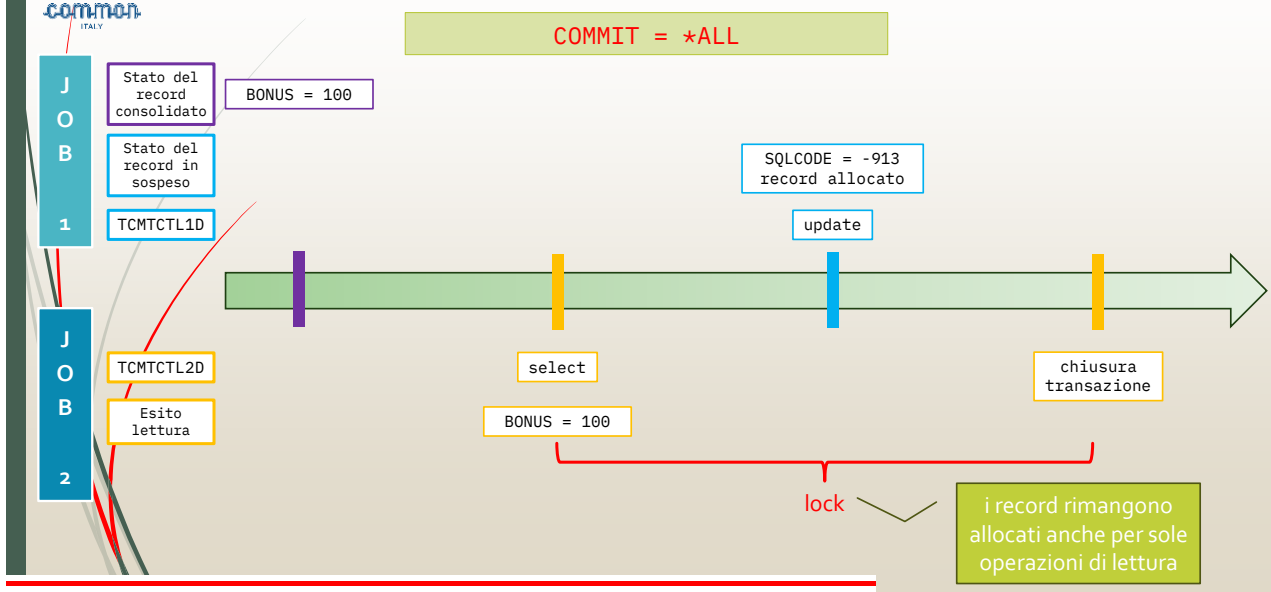

66

# *CHG vs *ALL

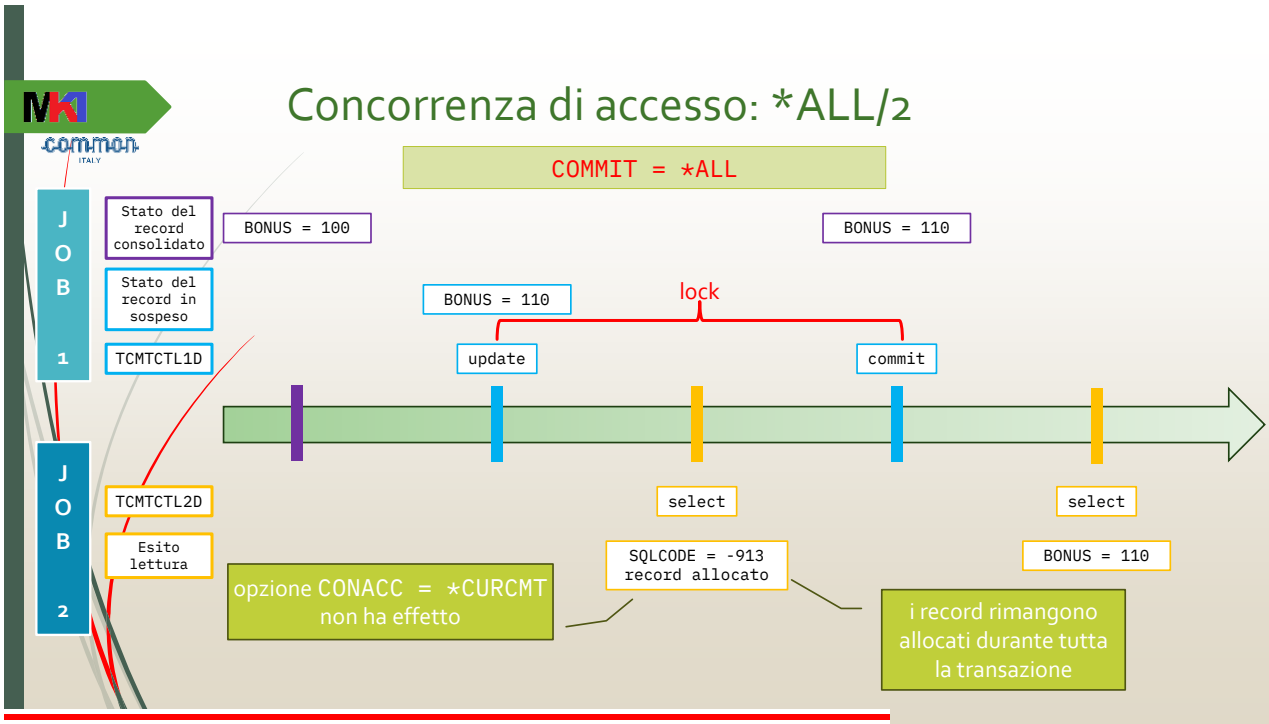
| Livello | Descrizione | Attivo | Letture sporche | Record fantasma | Allocazione record letti | Note |
|---|---|---|---|---|---|---|
| *ALL/RS | **LETTURA STABILE** (Read Stability) | ✓ | ✗ | ✓ | ✓ | *Garantisce che i record letti non possono essere modificati da un altro gruppo di attivazione e che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato → come RR*<br>Record "fantasma": a differenza di RR eseguendo più volte la stessa query possono comparire i record aggiunti in altri gruppi di attivazione |
| *CS/CS | **LETTURA SINCRONIZZATA** (Cursor Stability or Read Committed) | ✓ | ✗ | ✓ | ✗ | *Garantisce che ogni record modificato in un altro gruppo di attivazione non può essere letto fino a quando viene consolidato → come RR o RS*<br>A differenza di RR e RS i record letti possono essere modificati da altri gruppi di attivazione |
| *CHG/UR | **LETTURA NON SINCRONIZZATA** (Uncommitted Read) | ✓ | ✓ | ✓ | ✗ | Livello più basso di controllo di sincronia.<br>Consente di leggere record modificati in altri gruppi di attivazione anche se non ancora consolidate |

67

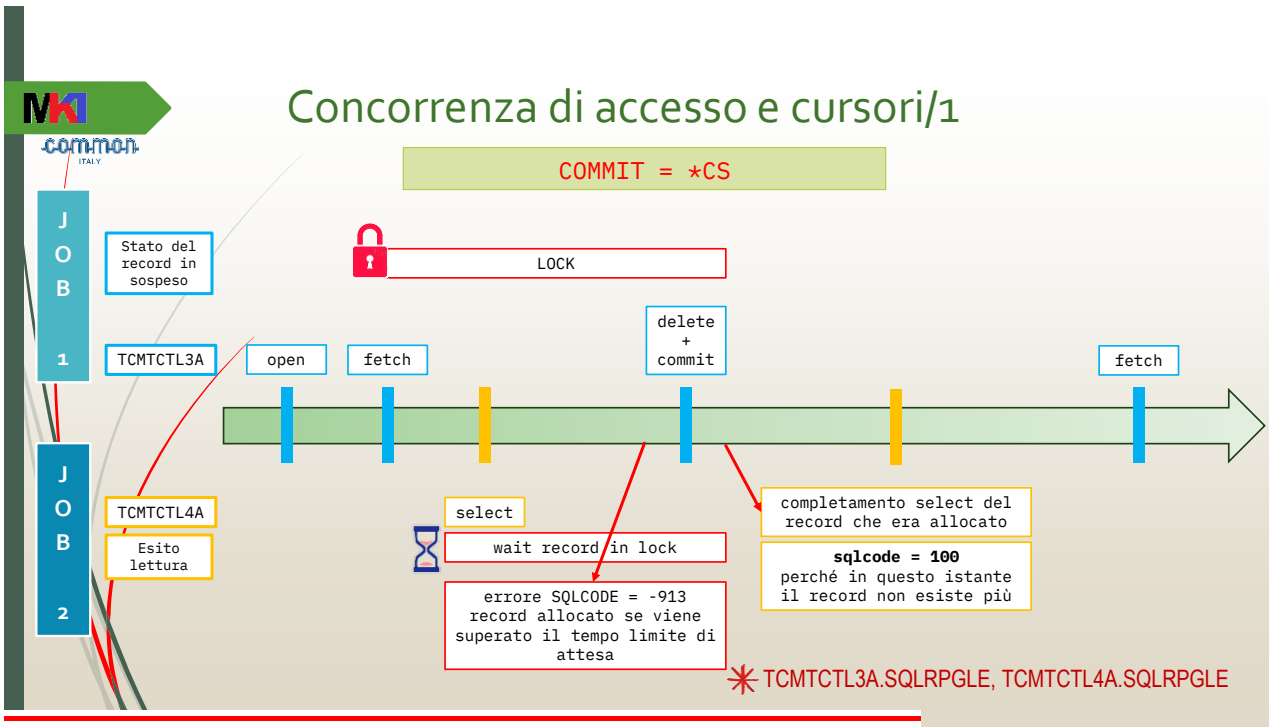
# Concorrenza di accesso: *ALL/1



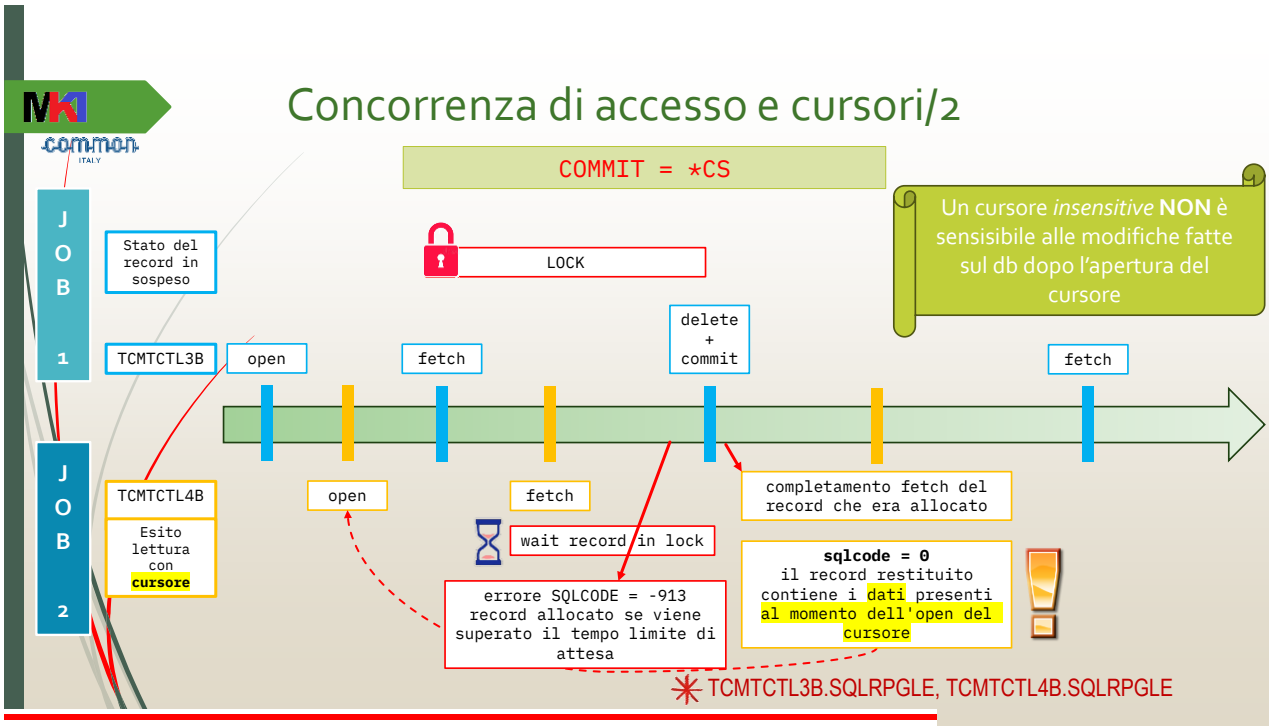

68


Concorrenza di accesso: *ALL/2
COMMIT = *ALL
JOB 1
Stato del record consolidato
BONUS = 100
BONUS = 110
Stato del record in sospeso
BONUS = 110
lock
TCMTCTL1D
update
commit
JOB 2
TCMTCTL2D
select
select
Esito lettura
SQLCODE = -913 record allocato
BONUS = 110
opzione CONACC = *CURCMT non ha effetto
i record rimangono allocati durante tutta la transazione

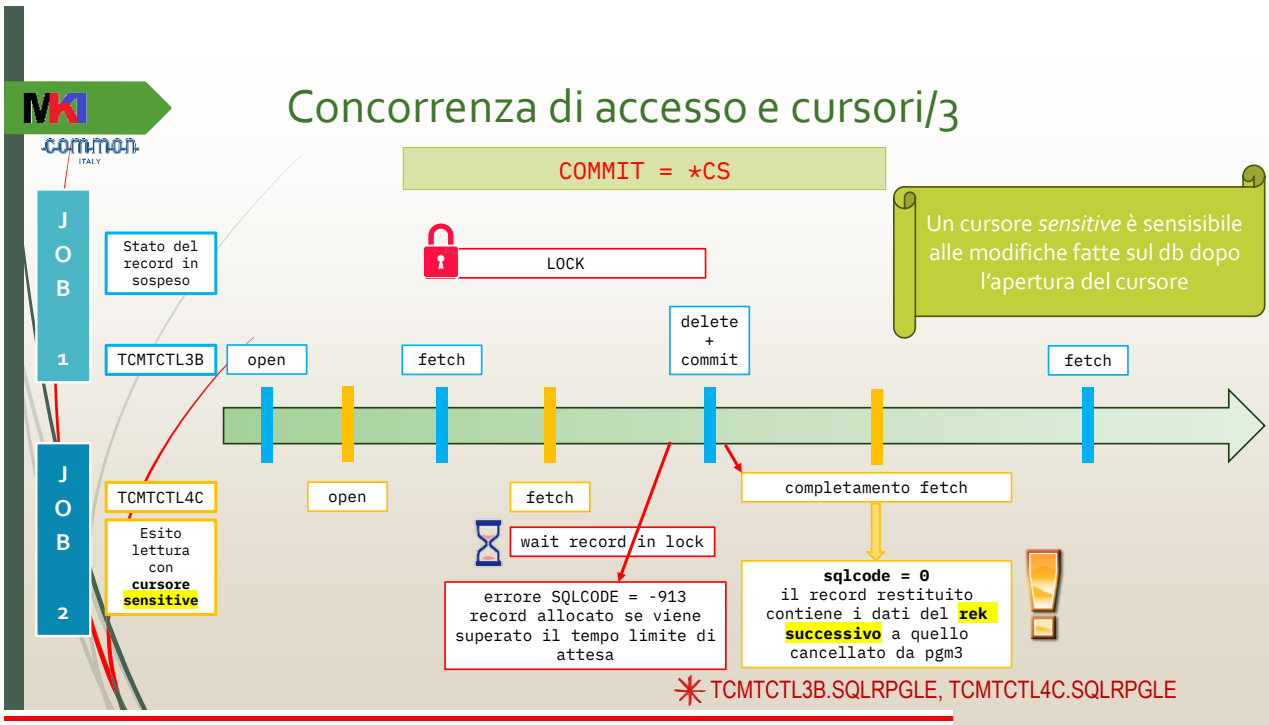
69


Concorrenza di accesso e cursori/1
COMMIT = *CS
JOB 1
Stato del record in sospeso
LOCK
delete + commit
TCMTCTL3A
open
fetch
fetch
JOB 2
TCMTCTL4A
select
completamento select del record che era allocato
Esito lettura
wait record in lock
sqlcode = 100 perché in questo istante il record non esiste più
errore SQLCODE = -913 record allocato se viene superato il tempo limite di attesa
TCMTCTL3A.SQLRPGLE, TCMTCTL4A.SQLRPGLE

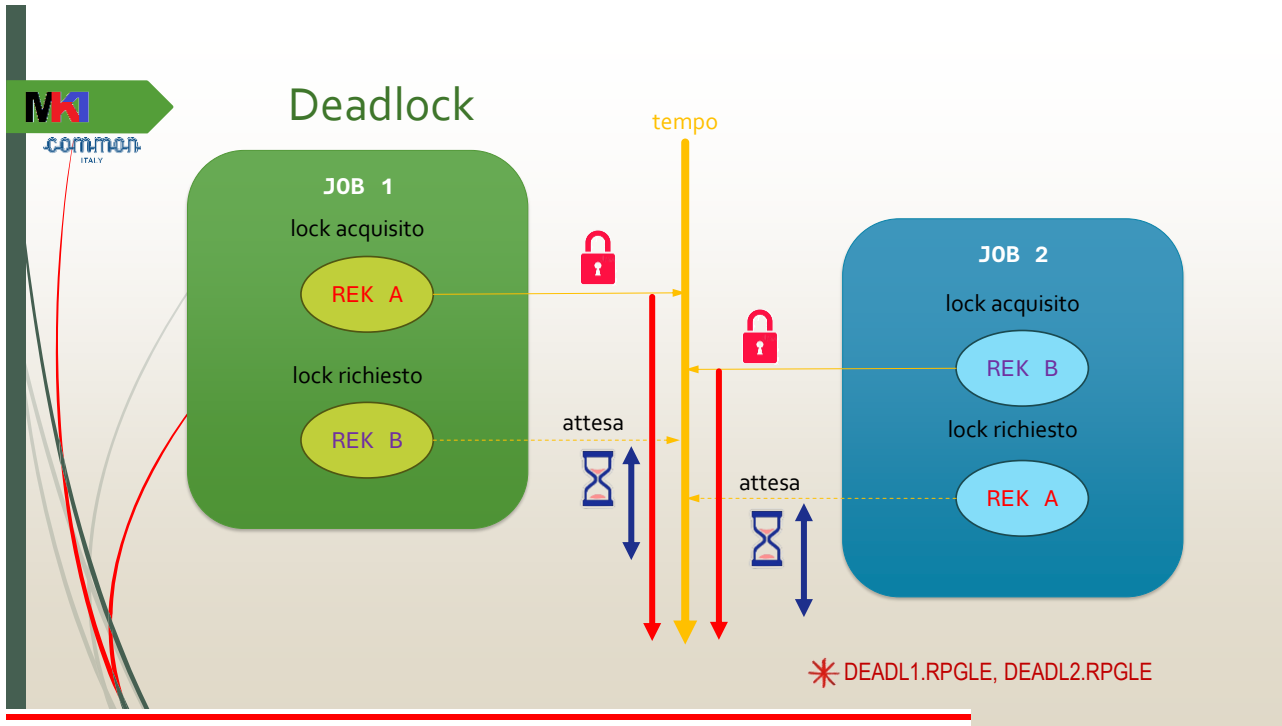
70


Concorrenza di accesso e cursori/2
COMMIT = *CS
Un cursore insensitive NON è sensisibile alle modifiche fatte sul db dopo l'apertura del cursore
JOB 1
Stato del record in sospeso
LOCK
TCMTCTL3B
open
fetch
delete + commit
fetch
JOB 2
TCMTCTL4B
Esito lettura con cursore
open
fetch
completamento fetch del record che era allocato
wait record in lock
errore SQLCODE = -913 record allocato se viene superato il tempo limite di attesa
sqlcode = 0 il record restituito contiene i dati presenti al momento dell'open del cursore
TCMTCTL3B.SQLRPGLE, TCMTCTL4B.SQLRPGLE

71


Concorrenza di accesso e cursori/3
COMMIT = *CS
Un cursore sensitive è sensisibile alle modifiche fatte sul db dopo l'apertura del cursore
JOB 1
Stato del record in sospeso
LOCK
TCMTCTL3B
open
fetch
delete + commit
fetch
JOB 2
TCMTCTL4C
Esito lettura con cursore sensitive
open
fetch
completamento fetch
wait record in lock
errore SQLCODE = -913 record allocato se viene superato il tempo limite di attesa
sqlcode = 0 il record restituito contiene i dati del rek successivo a quello cancellato da pgm3
TCMTCTL3B.SQLRPGLE, TCMTCTL4C.SQLRPGLE

72






## Controllo sincronia e cursori SQL/1

- Se si lavora con i cursori può essere necessario specificare la keyword **hold** per mantenere aperti i cursori

```
exec sql commit hold;
exec sql rollback hold;
```

*Esempio log con cursore e commit senza keyword hold*

```
Creato ODP.
Cursore CEMP aperto.
1 righe richiamate dal cursore CEMP.
1 righe aggiornate in EMPLOYEE in MK1SAMPLE.
1 righe richiamate dal cursore CEMP.
1 righe aggiornate in EMPLOYEE in MK1SAMPLE.
1 righe richiamate dal cursore CEMP.
1 righe aggiornate in EMPLOYEE in MK1SAMPLE.
ODP cancellato.
Commit completo.
```

Commit senza keyword hold chiude il cursore

Dopo commit alla successiva istruzione `fetch` restituisce sqlcode = `-501`

TCMTCS1.SQLRPGLE, TCMTCS2.SQLRPGLE

## Controllo sincronia e cursori SQL/2

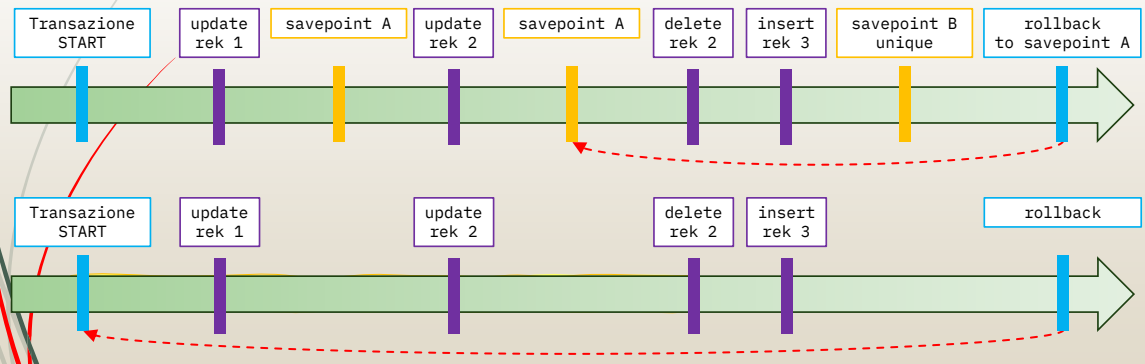
- durante il ciclo di lettura di un *cursore di sola lettura* l'istruzione `rollback hold` non modifica il posizionamento del cursore
- se il *cursore è di aggiornamento* un rollback eseguito durante il ciclo **riposiziona il cursore** al record sul quale si trovava a inizio del ciclo di commit, quindi dopo il rollback non è possibile eseguire un'istruzione `update/delete where current of...` e la successiva `fetch` riparte dal record letto all'inizio del ciclo di commit
- se il *cursore è di aggiornamento* un commit eseguito durante il ciclo consolida le modifiche pendenti e rilascia i lock dei record quindi non è possibile eseguire un'istruzione `update/delete where current of...`

✳ TCSRBK1A.SQLRPGLE, TCSRBK1B.SQLRPGLE, TCSRBK1C.SQLRPGLE

75

## Savepoint/1

- L'istruzione `savepoint` consente di identificare uno o più punti all'interno di un ciclo di commit dove è possibile ritornare con un'istruzione di rollback




76

## Savepoint/2

- `exec sql savepoint HERE`
  `on rollback retain cursors`
  p.es. il savepoint con nome HERE consente di poter eseguire un rollback a questo punto mantenendo aperto i cursori e mantenendo anche la posizione del cursore al momento in cui è stato salvato il savepoint
- si può tornare al savepoint HERE con l'istruzione SQL
  `exec sql rollback to savepoint HERE`

https://www.ibm.com/docs/en/i/7.6.0?topic=integrity-savepoints

TCSRBK3C.SQLRPGLE

77

## Savepoint/3

- Impostare un savepoint
  `exec sql savepoint nome [unique] [on rollback retain cursors]`
- Eseguire un rollback ritornando ad un savepoint salvato
  `exec sql rollback to savepoint nome`
- Eseguire un rollback ritornando all'ultimo savepoint
  `exec sql rollback to savepoint`
- Rilasciare un savepoint
  `exec sql release savepoint nome`
  in ogni caso il savepoint viene rilasciato al termine della transazione

78

## Gestione errori

- Codici di errore (program status code)
  - `00802`: Commitment control not active
  - `00803`: Rollback operation failed
  - `00804`: Error occurred on COMMIT operation
  - `00805`: Error occurred on ROLBK operation
- SQL code
  - `-774`, `-426`, `-30090`: Invalid Transaction Termination
  - `-7017`: SQL statements cannot be executed under commitment control, because commitment control is already active to another relational database
  - `-7007`: COMMIT or ROLLBACK is not allowed, because commitment control has not been started.
  - `-175`: The commit operation failed, because a resource in the unit of work was not able to commit its resources.
  - `-7061`: The object is in an invalid state for the operation

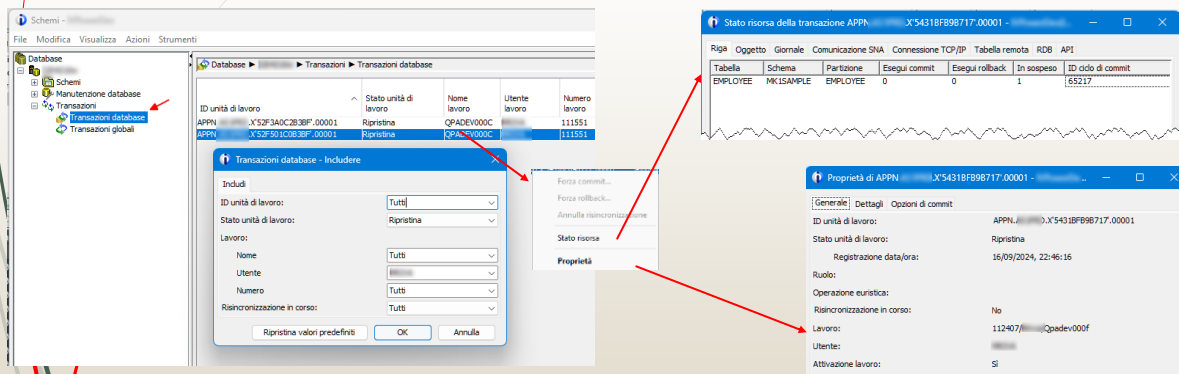


## Stato controllo sincronia

- Per visualizzare le informazioni sulla stato del controllo sincronia di un job è possibile
  - Menu gestione lavoro (Rich.sistema > opz. 3 Visual. lavoro corrente > opz. 16 Visualizzazione stato controllo convalida)
  - `WRKCMTDFN`: consente di visualizzare le informazioni del controllo di sincronia anche di altri lavori.
    P.es. tutti i lavori con transazioni pendenti
    `WRKCMTDFN JOB(*ALL) STATUS(*PENDING)`

# Stato controllo sincronia in ACS

Gestione schemi > Database > *Nome_sistema* > Transazioni > Transazioni database




N.B. occorre avere autorizzazione per il comando ADDPFTRG

Commitment control for everyone.sql



# Stato controllo sincronia con SQL

- DB_TRANSACTION_INFO: restituisce una riga per ogni definizioni di controllo sincronia
- DB_TRANSACTION_JOURNAL_INFO: restituisce le informazioni sulle risorse di giornale
- DB_TRANSACTION_OBJECT_INFO: restituisce informazioni sullo stato della transazione a livello di oggetto
- DB_TRANSACTION_RECORD_INFO: restituisce informazioni sullo stato della transazione a livello di record

Commitment control for everyone.sql

## Stato controllo sincronia

Informazioni visualizzate in base all'ambito della definizione del controllo di sincronia

| | commitment definition | activation group | job | thread |
|---|---|---|---|---|
| JOB-LEVEL | *JOB | == | nome job | *NONE |
| ACTIVATION-GROUP-LEVEL | nome activation group | nome activation group | nome job | *NONE |
| TRANSACTION -SCOPED | *TNSOBJ | == | nome job | thread |
| EXPLICITLY-NAMED[1] | nome definizione (Q*) | == | nome job | *NONE |

[1] possono essere avviati solo dal sistema operativo per gestire sue transazioni indipendenti dall'applicazione

83

## Nome definizione controllo sincronia

| Activation group | Ambito | Nome |
|---|---|---|
| qualsiasi | job | *JOB |
| default activation group | activation group | *DFTACTGRP |
| user-named activation group | activation group | nome activation group |
| system-named activation group | activation group | nome activation group |
| == | explicitly named | Q* |
| == | transaction | *TNSOBJ |

84

## Identificatore del ciclo di commit

- Un ciclo di commit è il tempo che intercorre tra due confini di commit
- Il ciclo di commit è identificato dal **numero di sequenza della voce di giornale C SC** (start commit cycle)
- L'identificatore del ciclo di commit è **presente in ogni voce di giornale** scritta durante il ciclo di commit

85

## ID transazione

- L'ID di una transazione o LUW è una stringa così composta

APPN.*sysname*.X'48953C039E92'.00001

- la porzione evidenziata in azzurro è diversa per ogni *definizione di controllo sincronia*
- all'interno della stessa definizione di controllo sincronia ad ogni commit/rollback la porzione finale evidenziata in rosso si incrementa identificando il *ciclo di commit*

86

## Voci di giornale per controllo sincronia/1

Voci di giornale per cicli di controllo sincronia

- C BC: begin commitment control -> STRCMTCTL
- C **SC**: start commit cycle -> al primo record modificato
- C PC: Preparazione blocco di commit -> inizio commit implicito
- C **CM**: commit -> commit concluso con successo
- C R1: inizio del rollback
- C **RB**: rollback -> rollback concluso con successo
- C CN: fine del rollback
- C LW: fine transazione -> quando si conclude il commit/rollback
- C EC: fine commitment control -> ENDCMTCTL

Per le voci CM e RB viene indicato se il commit è implicito (contrassegno = 2) o esplicito (contrassegno = 0)

87

## Voci di giornale per controllo sincronia/2

Voci di giornale per operazioni relative ai savepoint:

- C SB: start of savepoint -> creazione di un SQL savepoint
- C SQ: release of savepoint -> quando viene rilasciato un SQL savepoint
- C SU: rollback of savepoint -> quando viene eseguito un rollback di un savepoint

Non vengono scritte su giornale se la variabile di ambiente
`QTN_JRNSAVPT_MYLIB_MYJRN = *NO`
con MYLIB e MYJRN rispettivamente nome libreria e nome giornale

88

## Voci di giornale per controllo sincronia/3

Voci di giornale per aggiornamenti dei record a seguito di un'operazione di rollback.

- R BR: immagine precedente del record aggiornato per rollback
- R DR: record cancellato per rollback
- R UR: immagine successiva del record aggiornato per rollback

89

## Gestire i giornali con SQL

- Proprietà del giornale: JOURNAL_INFO
- Proprietà del ricevitore: JOURNAL_RECEIVER_INFO
- Pulizia ricevitori: DELETE_OLD_JOURNAL_RECEIVERS
- Tipi voci di giornale: JOURNAL_CODE_INFO
- Visualizzazione voci di giornale: DISPLAY_JOURNAL

Creare SQL Views per interpretare QSYS2.DISPLAY_JOURNAL
(FAQ400, 10-mar-2023, Roberto De Pedrini)
Procedura Create_Display_Journal_Table_View

Tool EXPJRNE
(Thomas Raddatz, 10-apr-2025)

Plug-in iSphere di RDi
(Thomas Raddatz)

Commitment control for everyone.sql

90

## Perché lo si teme?

- de che tutti i file siano ati sullo stesso giornale. Quindi s er timore di dec to di perf ce e di occ e spazio di
- prima d V er inoso l'uso o sincronia i i
- prima di ILE lo di sincronia agiv dell'activa
- **poich ono re i su giornale i atte e nella man ne di oggetti e librerie**
- **essità del disegno a tivo per definire correttamente ansazioni e gli activatio**

macchina

uomo

91

92

## Glossario del controllo di sincronia

| It | En |
|---|---|
| transazione | logical unit of work (LUW) |
| ciclo di commit | commit cycle |
| livello di isolamento | isolation level |
| confine della transazione | boundary |
| ambito | scope |
| definizione di controllo sincronia | commitment definition |
| commitment control | controllo di sincronia |

93

## Bibliografia


- Database commitment control 7.6: https://www.ibm.com/docs/en/ssw_ibm_i_76/pdf/rzakjpdf.pdf
- Documentazione IBM commitment control 7.6: https://www.ibm.com/docs/en/i/7.6.0?topic=database-commitment-control
- Documentazione IBM SQL reference 7.6: https://www.ibm.com/docs/en/i/7.6.0?topic=reference-sql
- Documentazione IBM SQL Programming 7.6 - Commitment control: https://www.ibm.com/docs/en/i/7.6.0?topic=integrity-commitment-control
- Looking for Commitment part 1, di Paul Tuohy, 19-giu-2019, https://www.itjungle.com/2019/06/19/guru-classic-looking-for-commitment-part-1/
- Looking for Commitment part 2, di Paul Tuohy, 17-lug-2019, https://www.itjungle.com/2009/03/18/fhg031809-story01/
- Looking for Commitment part 3, di Paul Tuohy, 14-ago-2019, https://www.itjungle.com/2019/08/14/guru-classic-looking-for-commitment-part-3/
- Soft Commit: Worth a Try on IBM i5/OS V5R4, di Hernando Bedoya, 31-lug-2006, https://www.redbooks.ibm.com/abstracts/tips0623.html#4


94

## Riferimenti

- E-mail aziendale: marco.riva@ivolution.it
- Blog: www.markonetools.it
- E-mail blog: info@markonetools.it
- Linkedin: www.linkedin.com/in/marcoriva-mk1
- Twitter: @MarcoRiva73
- Facebook: https://www.facebook.com/markonetools/
- YouTube: youtube.com/@markonetools

95

## Disclaimer

DISCLAIMER 2022 MarkOneTools

**Questa pubblicazione è fornita "as is" senza garanzia di alcun tipo.**

Tutti i riferimenti in questo documento a siti web sono forniti solo per comodità e non possono essere considerati come approvazione di tali siti web. I materiali di tali siti web non fanno parte del materiale di questo documento e l'uso di tali siti web è a proprio rischio.

Tutti i marchi appartengono ai rispettivi proprietari.

Le informazioni contenute nella presente documentazione possono essere riservate e sono, comunque, **destinate esclusivamente ai partecipanti al corso**. I contenuti del documento sono stati redatti con la massima cura e diligenza, e sottoposti ad un accurato controllo. **Si declina tuttavia ogni responsabilità**, diretta e indiretta, nei confronti degli utenti e in generale di qualsiasi terzo, **per eventuali imprecisioni, errori, omissioni derivanti dai suddetti contenuti. Nessuna parte di questa pubblicazione può essere riprodotta o trasmessa in qualsiasi forma o per qualsiasi scopo senza la preventiva ed espressa autorizzazione del proprietario.**

Questo documento contiene esempi di programmi in linguaggio sorgente, i quali illustrano le tecniche di programmazione. E' possibile copiare, modificare e distribuire questi esempi in ogni forma allo scopo di sviluppo ed utilizzo. Questi esempi non sono stati sottoposti a un processo di test sotto tutte le condizioni. Perciò non si garantisce l'affidabilità e la funzionalità completa dei programmi di esempio.

96

## Personal disclaimer

DISCLAIMER 2022 MarkOneTools

**Ho cercato in tutti i modi di produrre una documentazione ben fatta, precisa ed affidabile.**

**Le informazioni tecniche derivano dallo studio dei manuali IBM, della letteratura disponibile nel mondo del web, dalla pratica ed esperienza personale, dalla tradizione orale e dal confronto con persone di ogni livello.**

**Le opinioni sono opinioni. Ognuno ha le proprie:**

- **le mie opinioni sono mie. Non dell'azienda per cui lavoro, non di IBM**
- **le mie opinioni non sono necessariamente corrette**
- **ma d'altronde nemmeno le tue lo sono**

97

## Esempi da verificare

- Job con liv *CS e job con *NONE. Il job 2 legge le modifiche in sospeso oppure quelle precedenti consolidate sul DB?
- Come viene deciso se il cursore è sensitive/insensitive?
- Soft commit
- Commitment control con db remoto

98